Convolutional Neural Networks

Hrachya Asatryan

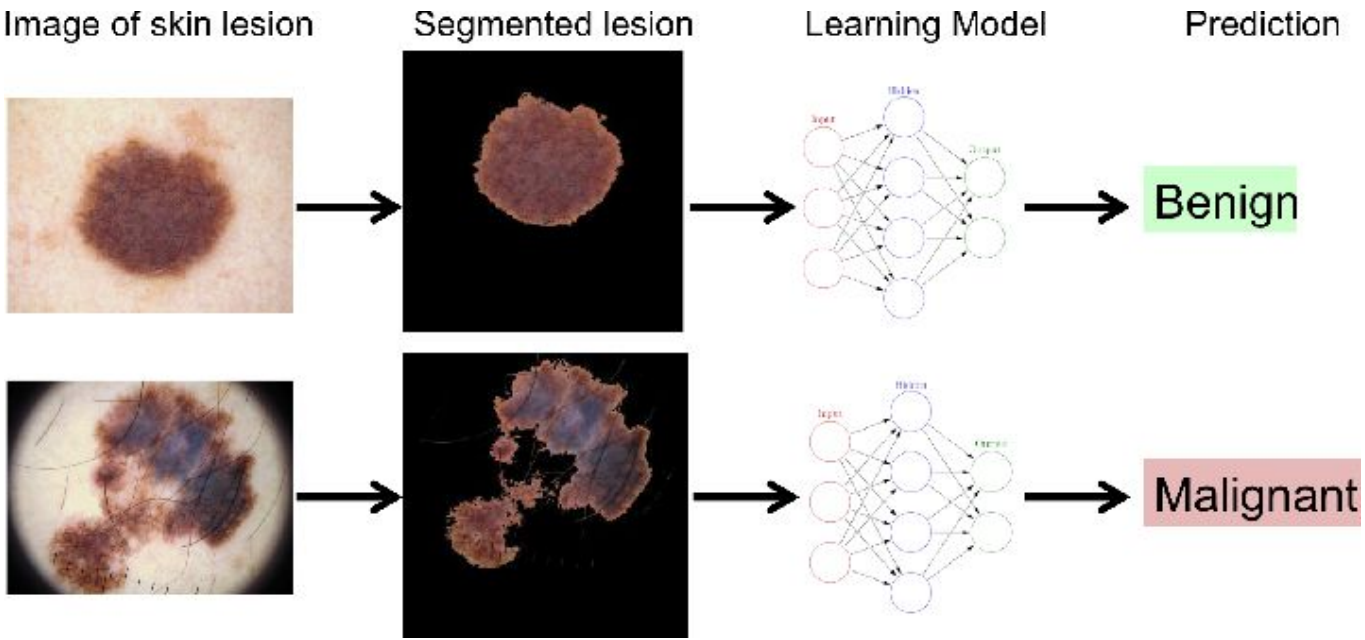
Computer Vision

Computer vision (CV) is a field of machine learning (ML) that uses machine learning and neural networks to teach computers and systems to derive meaningful information from digital images, videos and other visual inputs—and to make recommendations or take actions when they see defects or issues.

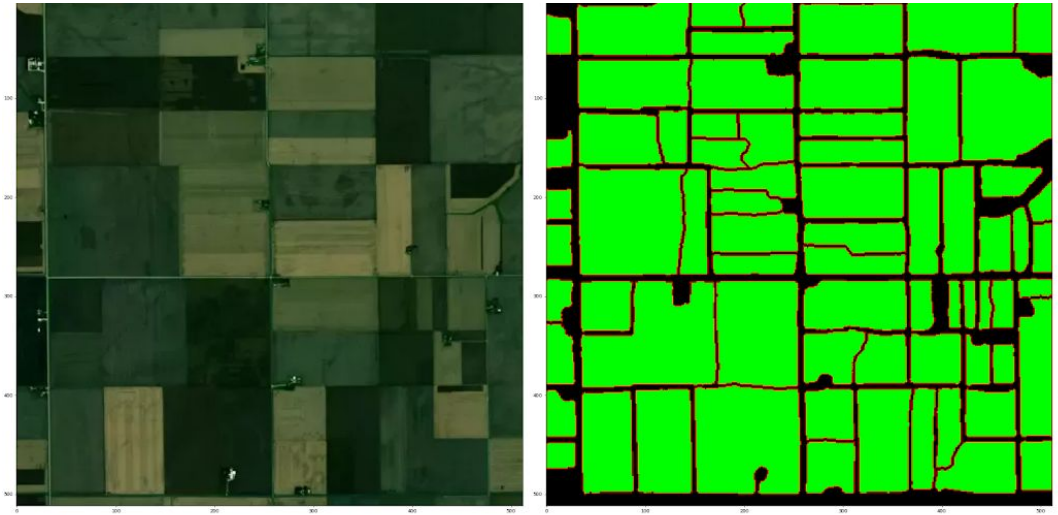
Computer vision tasks include methods for acquiring, processing, analyzing and understanding digital images, and extraction of high-dimensional data from the real world in order to produce numerical or symbolic information, e.g. in the forms of decisions.

Applications of CV

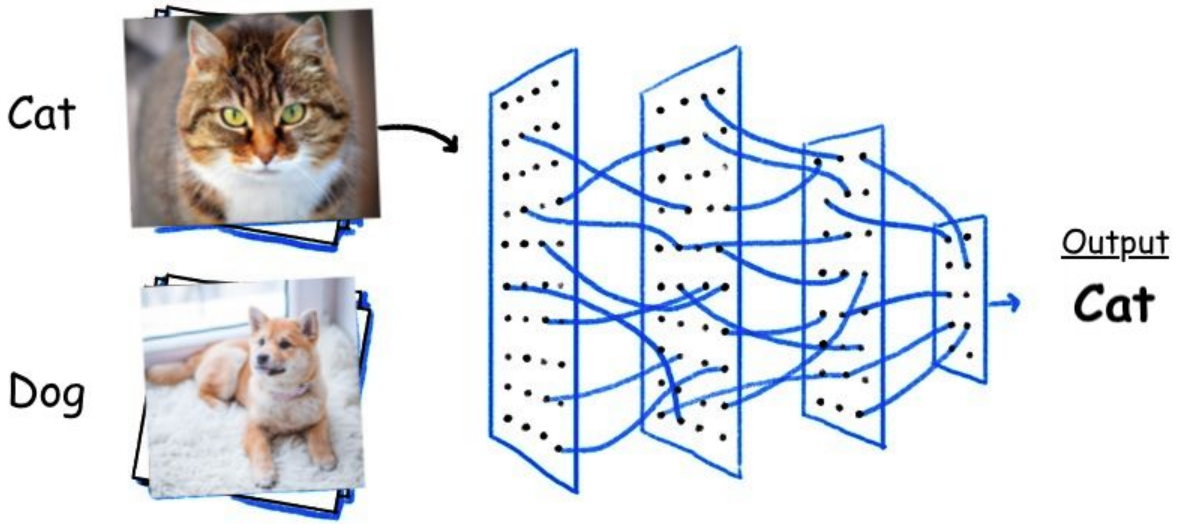
Applications



Applications



Applications



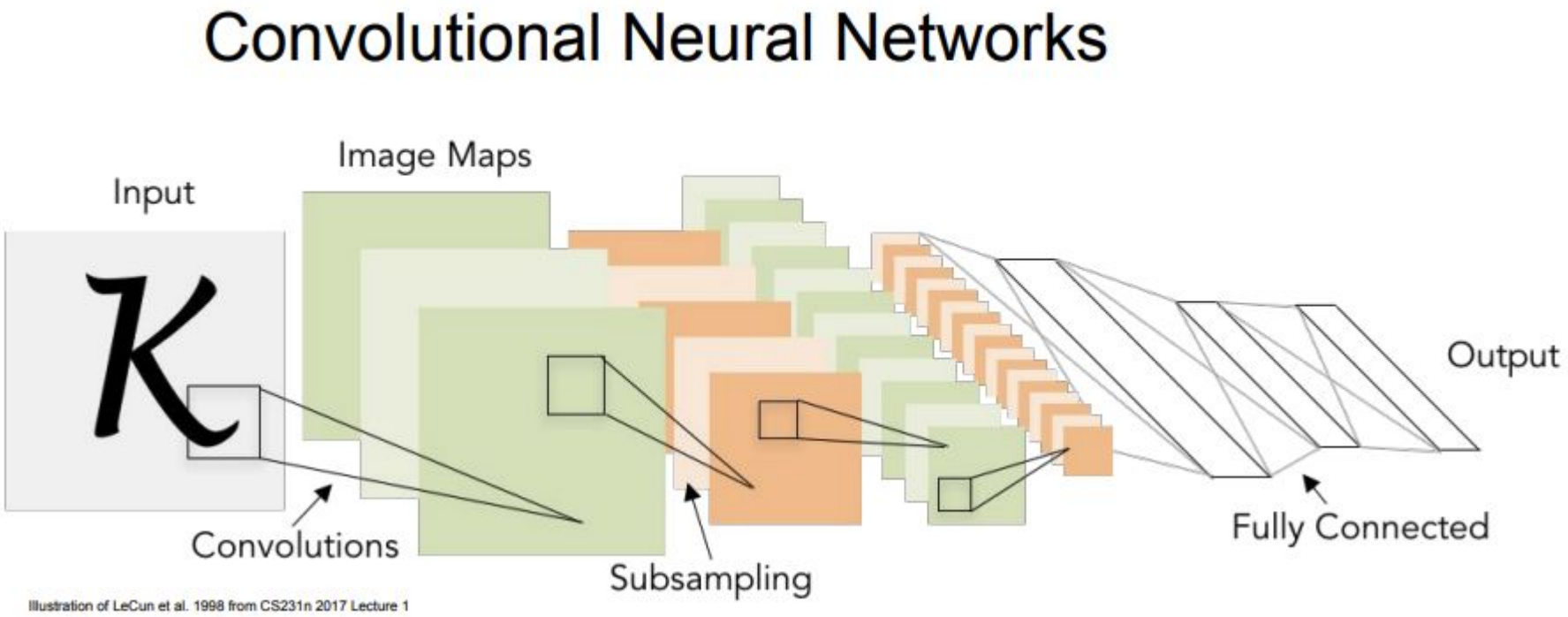
Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep learning algorithm designed specifically to analyze visual data. Inspired by the organization of the animal visual cortex, CNNs are adept at recognizing patterns in images through a hierarchical structure of interconnected layers. Its key components are:

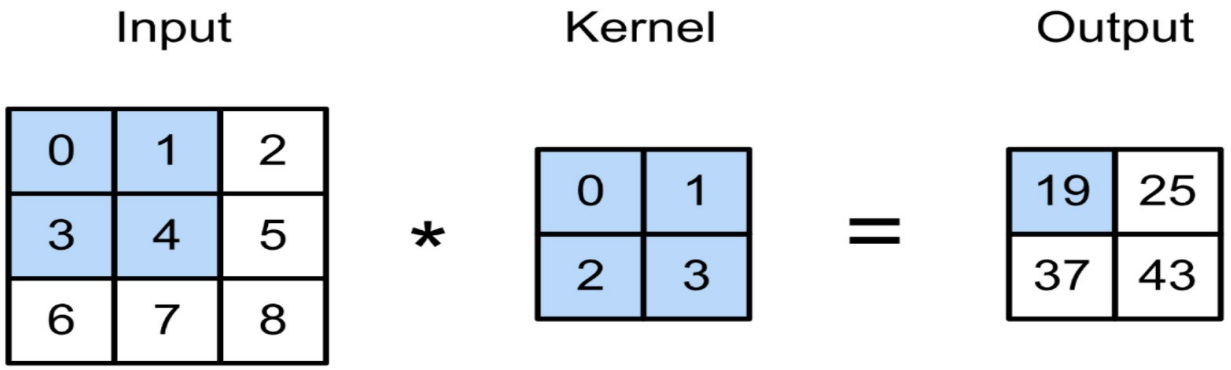
- Convolutional Layers: These layers apply filters to input images, extracting features such as edges, textures, and shapes.
- Pooling Layers: Pooling layers downsample the feature maps, reducing computational complexity while retaining important information.
- Fully Connected Layers: These layers perform classification based on the features extracted by the preceding layers.

CNNs are highly effective in tasks like image classification, object detection, and segmentation, making them essential tools in computer vision and beyond.

Visualization

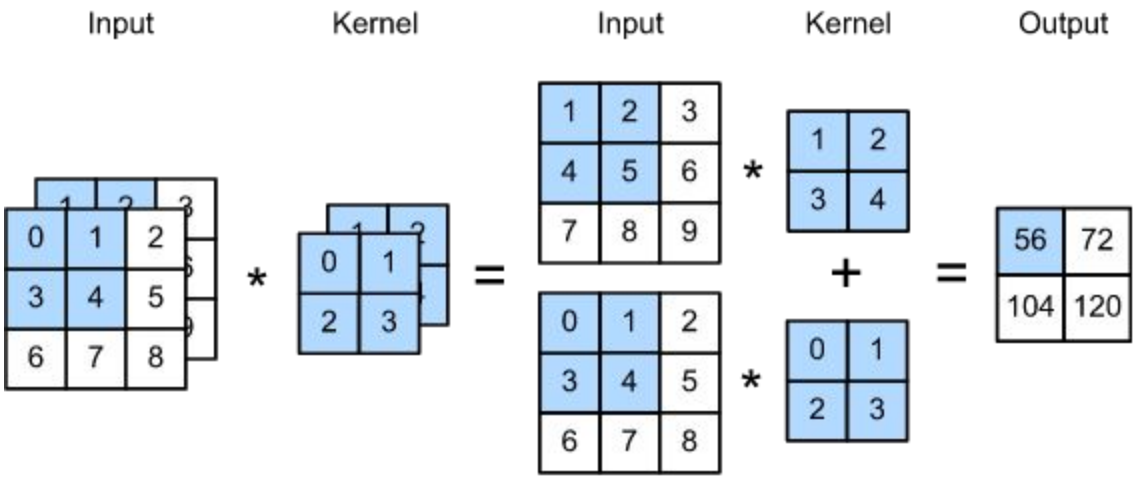


Convolutions



$$
\begin{aligned}
0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 &= 19, \\
1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 &= 25, \\
3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 &= 37, \\
4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 &= 43.
\end{aligned}
$$

Convolutions



Examples

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

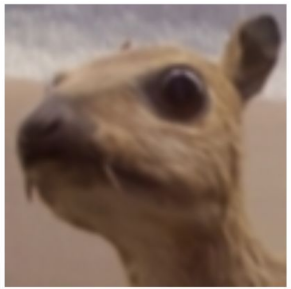
Edge Detection

(wikipedia)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

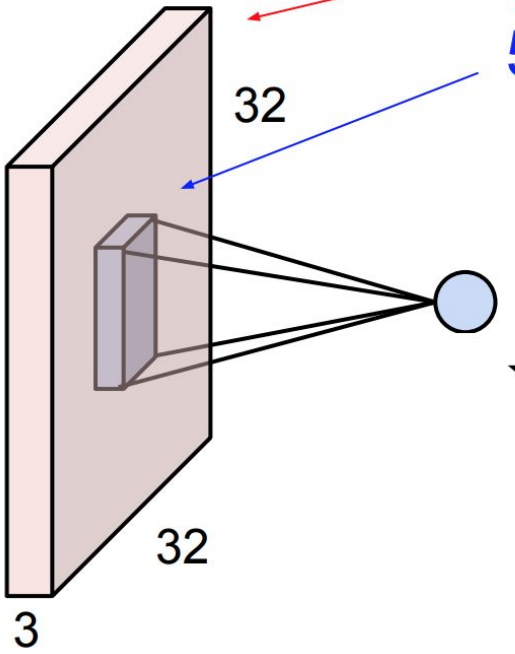
Sharpen

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



Gaussian Blur

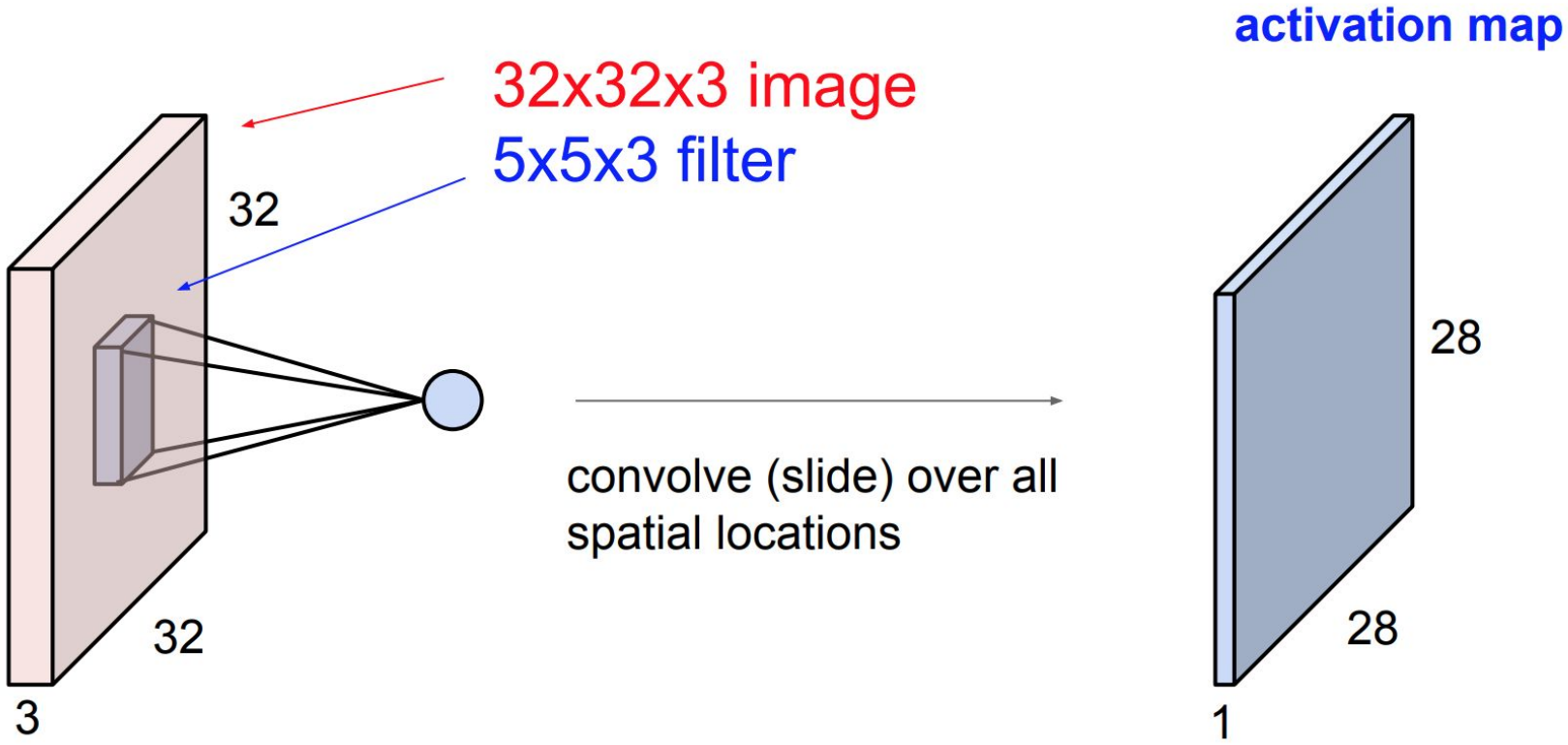
Convolutional Layer



1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

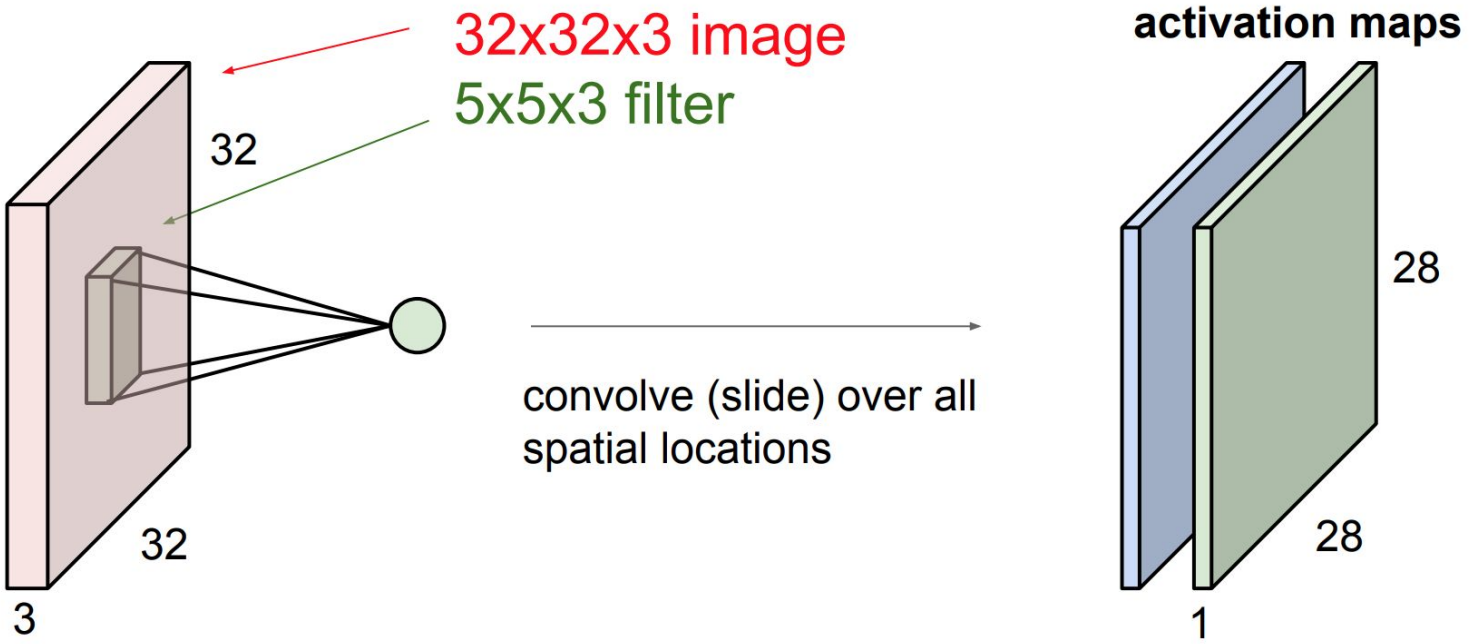
$$w^T x + b$$

Convolutional Layer



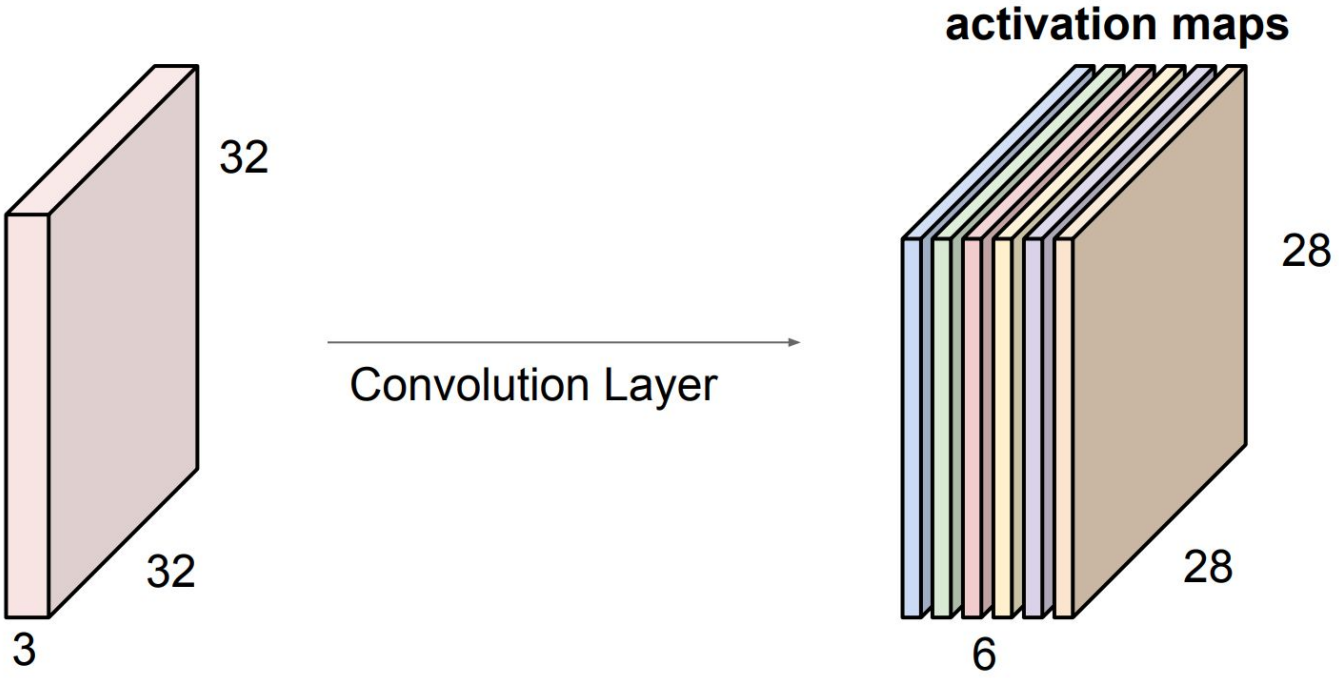
Convolutional Layer

consider a second, green filter



Convolutional Layer

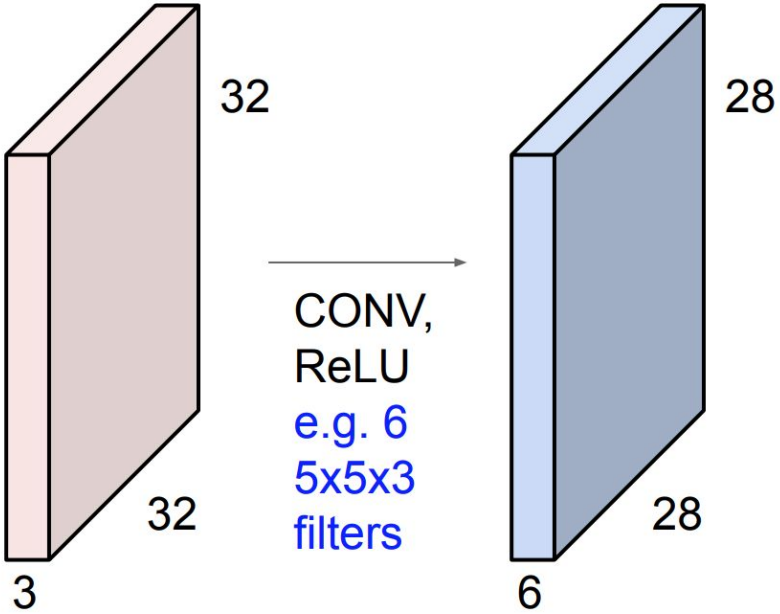
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a "new image" of size 28x28x6!

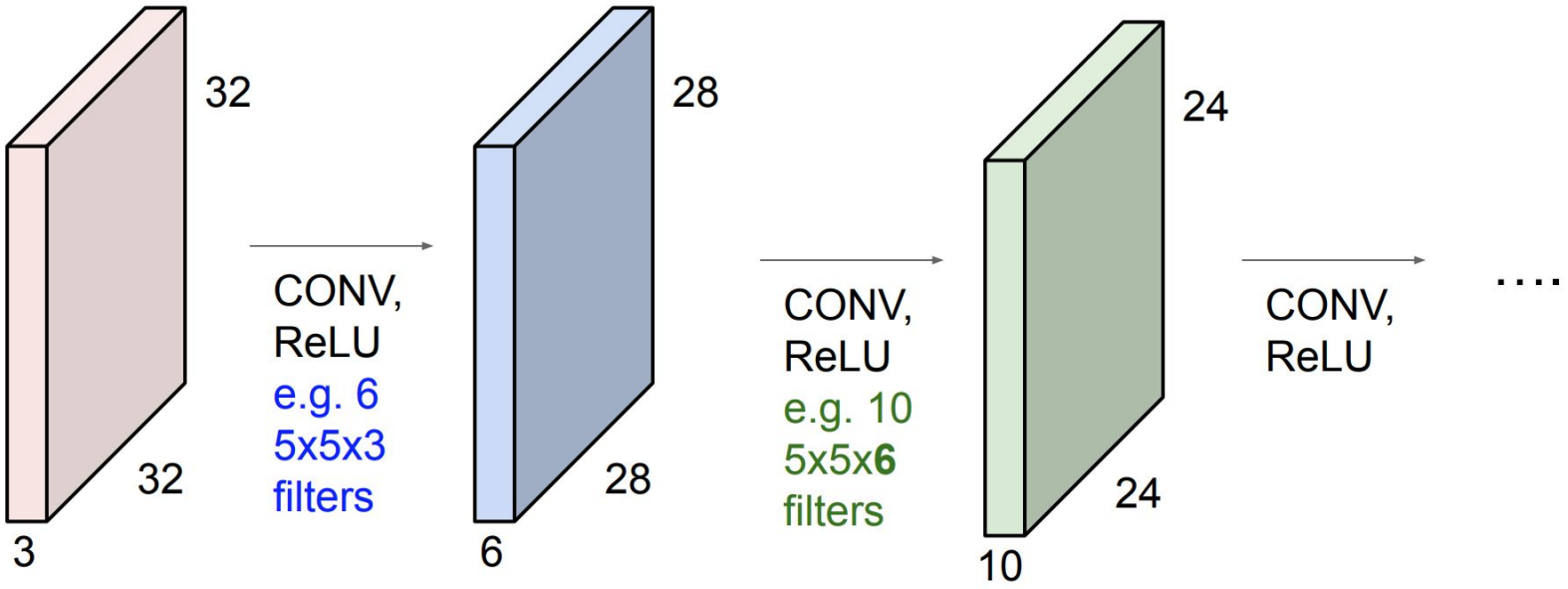
ConvNet

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



ConvNet

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



Padding

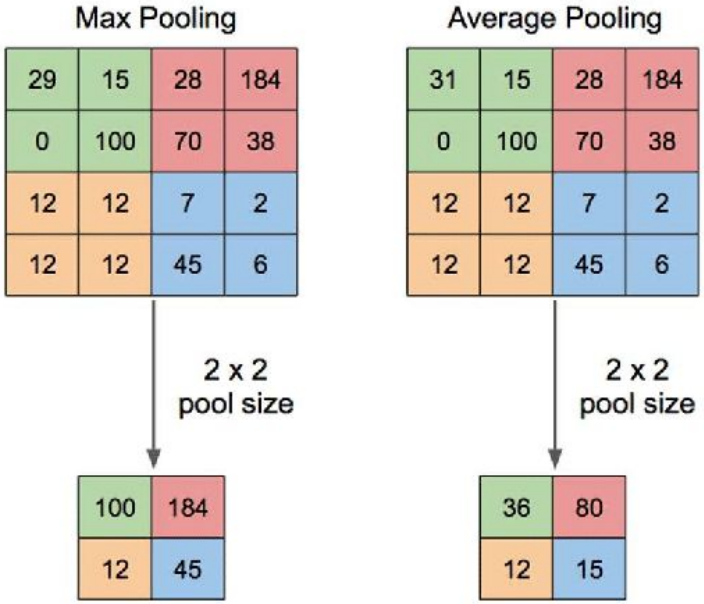
0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

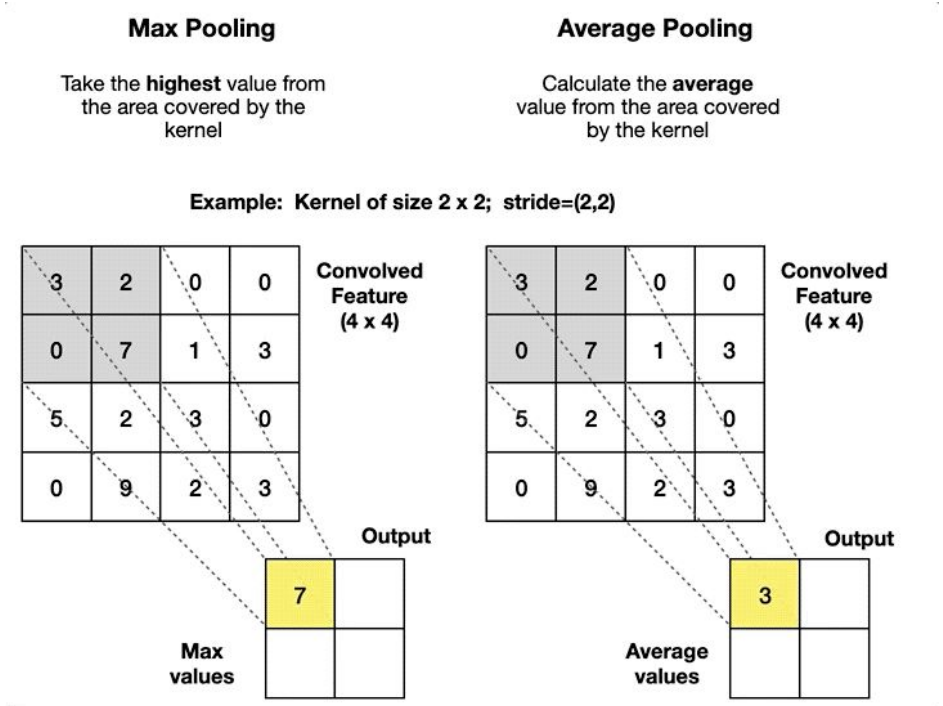
Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

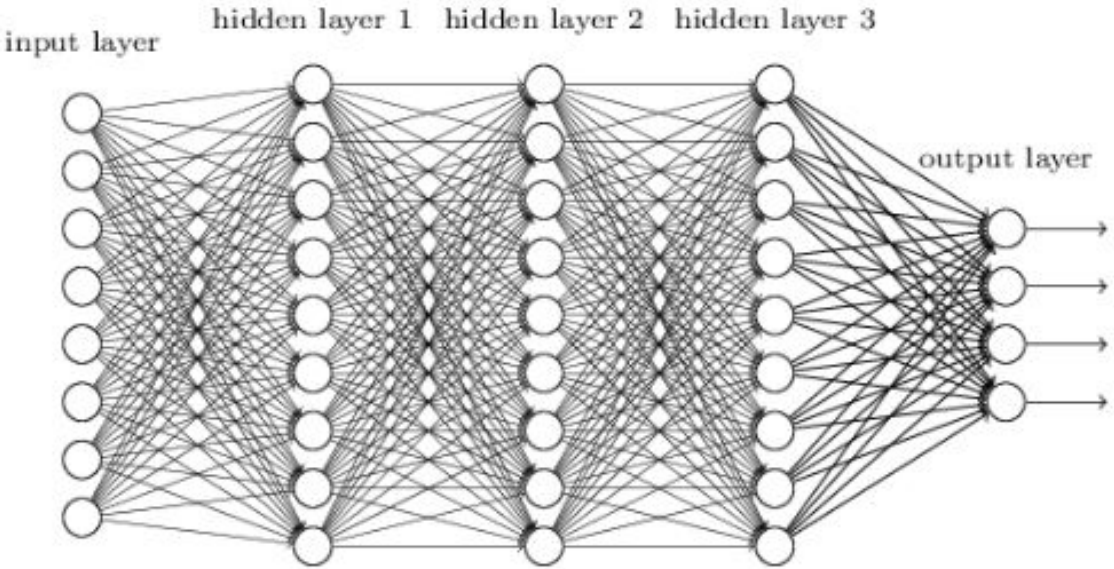
Pooling


Max Pooling
29 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
100 184
12 45
Average Pooling
31 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
36 80
12 15


Max Pooling
Take the highest value from the area covered by the kernel
Average Pooling
Calculate the average value from the area covered by the kernel
Example: Kernel of size 2 x 2; stride=(2,2)
3 2 0 0
0 7 1 3
5 2 3 0
0 9 2 3
Convolved Feature (4 x 4)
Output
7
Max values
3 2 0 0
0 7 1 3
5 2 3 0
0 9 2 3
Convolved Feature (4 x 4)
Output
3
Average values

Convolutional Layer

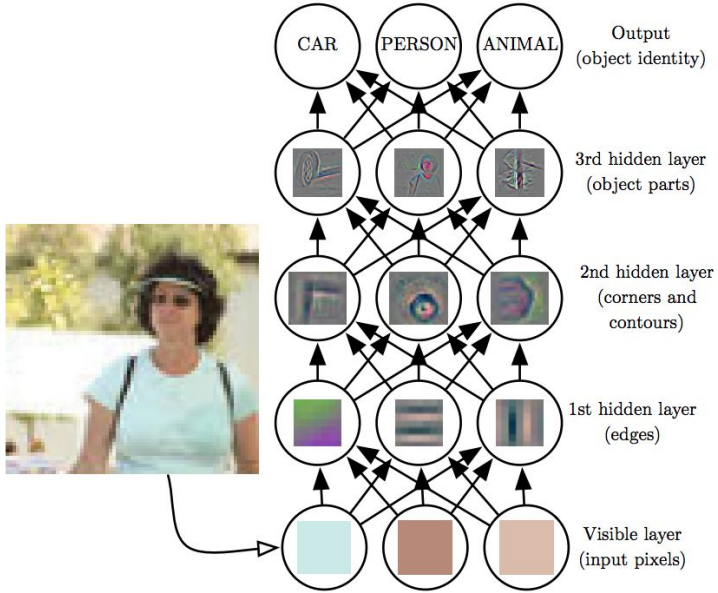
Consider this fully connected model



From this fully connected model, do we really need all the edges? Can some of these be shared?

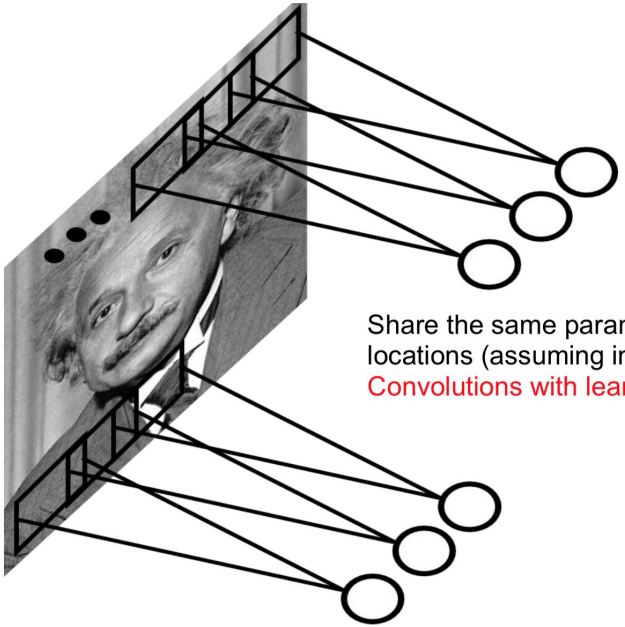
Feature Learning

Through successive layers of convolutional and pooling operations, CNNs can capture increasingly complex features, starting from simple edges and textures to more abstract concepts like shapes and objects.



Weight Sharing

Parameter sharing greatly reduces the number of parameters compared to fully connected networks, making CNNs more efficient to train and less prone to overfitting. Additionally, it helps CNNs generalize better to variations in input data, such as translation, rotation, and scale changes.

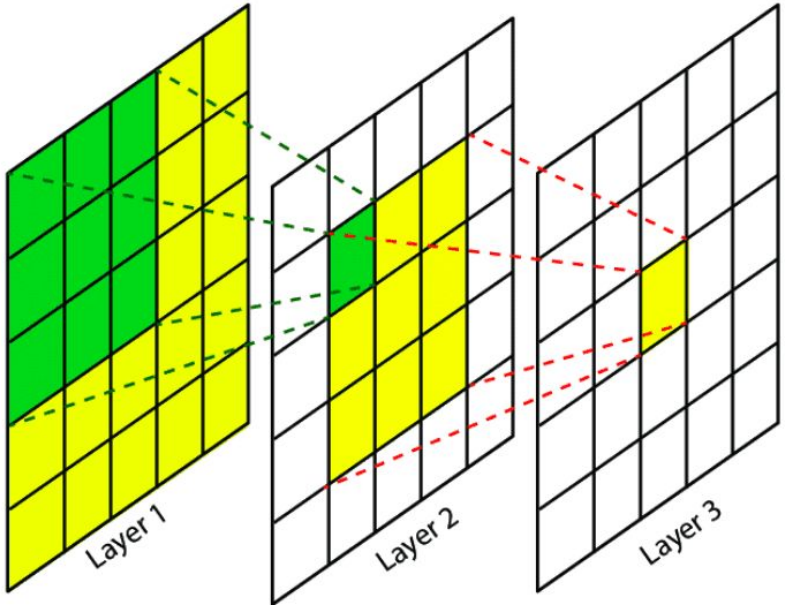


Share the same parameters across different locations (assuming input is stationary):
Convolutions with learned kernels

Ranzato

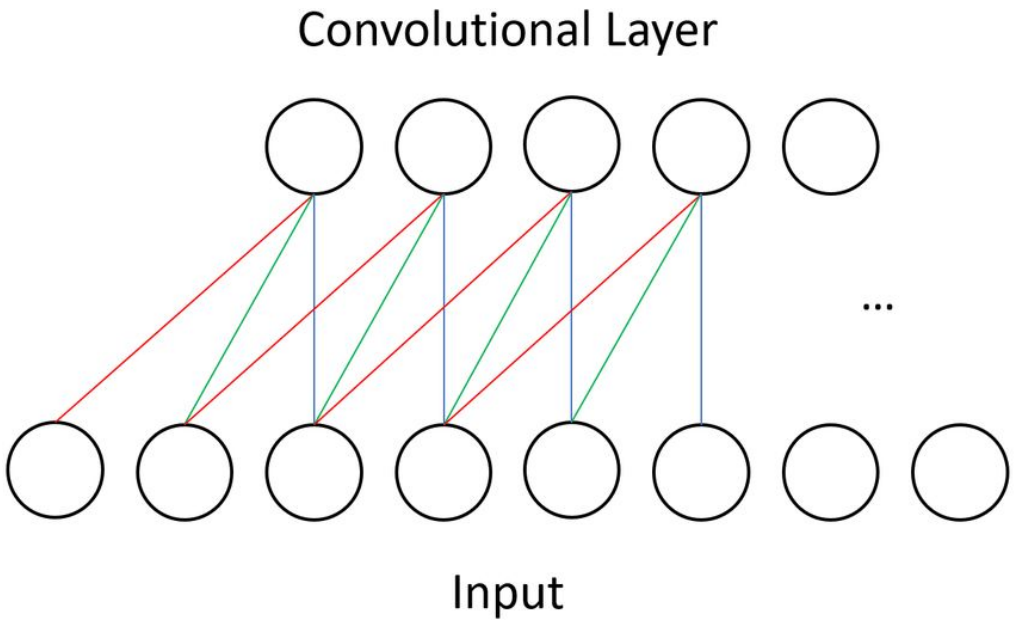
Local Receptive Field

By focusing on small regions of the input at a time and sharing parameters across these regions, CNNs can effectively capture local patterns and spatial relationships in the data. This allows CNNs to achieve translation invariance, meaning they can recognize objects regardless of their position within the input image.

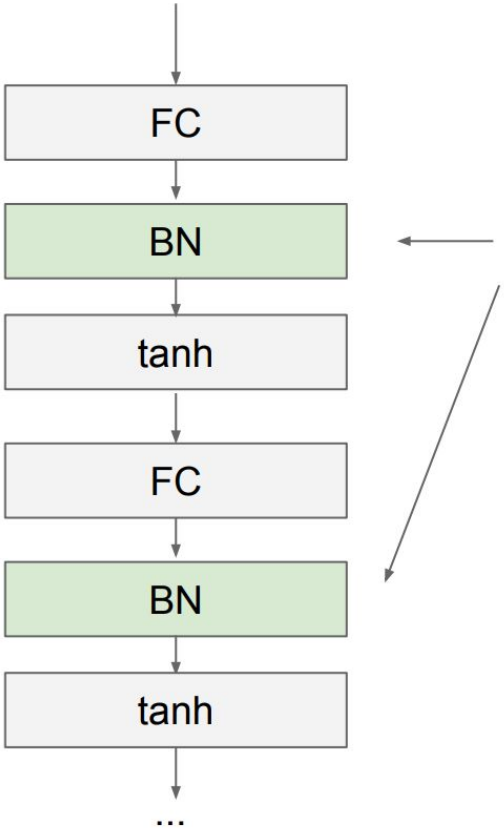


Sparse connectivity

In CNNs, each neuron in a convolutional layer is connected only to a small subset of neurons in the previous layer, determined by the size of the convolutional filters. This sparse connectivity reduces the computational complexity of the network while still allowing it to capture meaningful patterns in the data.



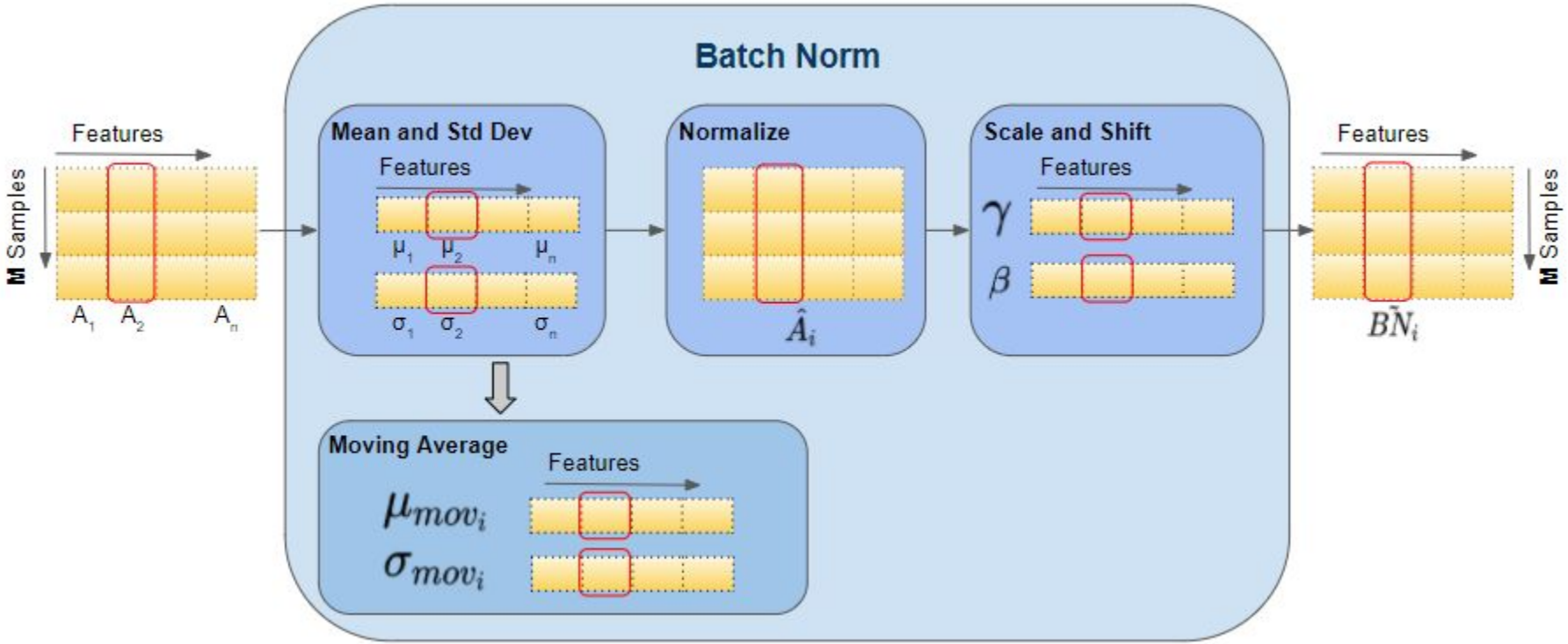
Batch Normalization



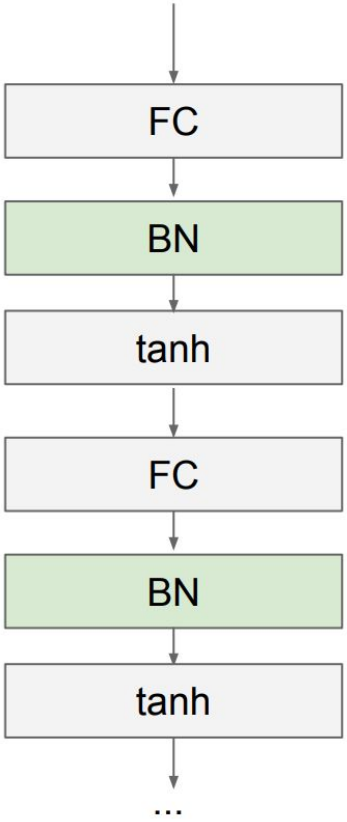
Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

Batch Norm


Batch Norm
Features
M Samples
A1 A2 An
Mean and Std Dev
Features
μ1 μ2 μn
σ1 σ2 σn
Normalize
$\hat{A}_i$
Scale and Shift
Features
γ
β
Features
M Samples
$\tilde{BN}_i$
Moving Average
Features
μ_{mov_i}
σ_{mov_i}

Batch Norm



- Makes deep networks **much** easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!
- Behaves differently during training and testing: this is a very common source of bugs!

The Problem with Batch Norm

- **Key Concept:** Batch Norm relies on accurate statistics (mean/variance) calculated from the current mini-batch.
- **The Issue:** If your batch size is small (e.g., 1 or 2 images per GPU because the model is huge or image resolution is high, like in Medical Imaging or Object Detection), the statistics are noisy. This leads to high training error.

Alternative Normalization Methods

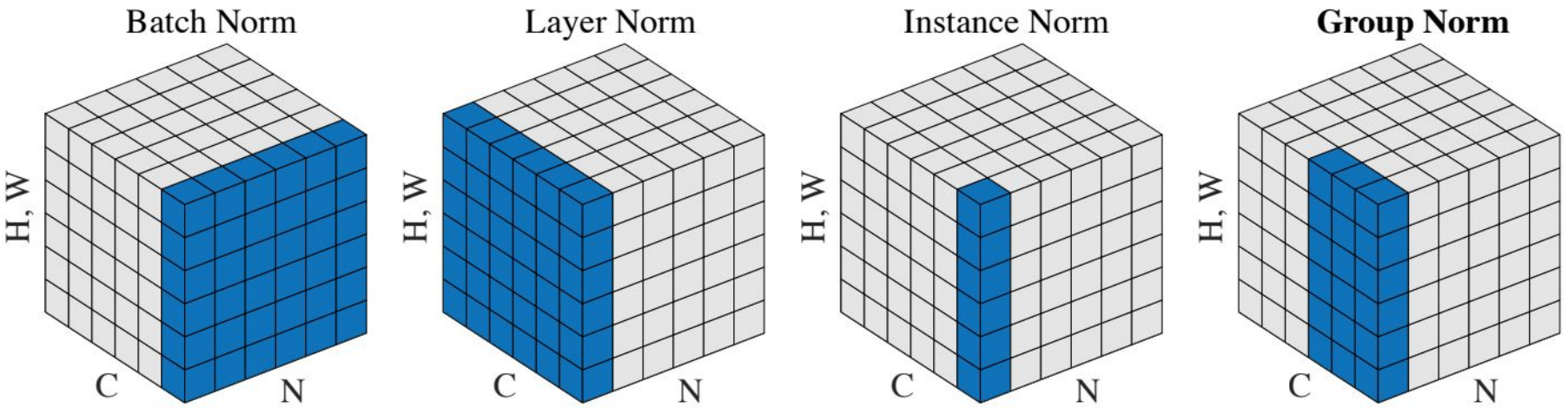
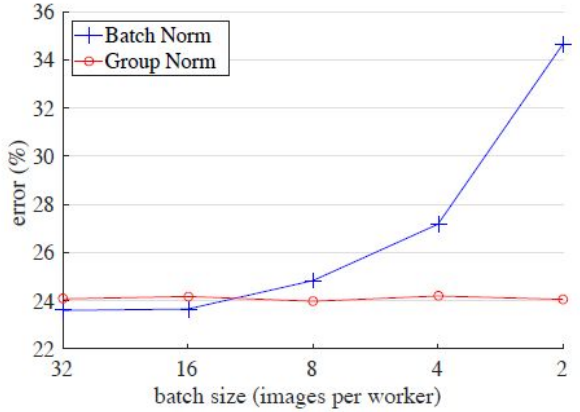


Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Group Norm

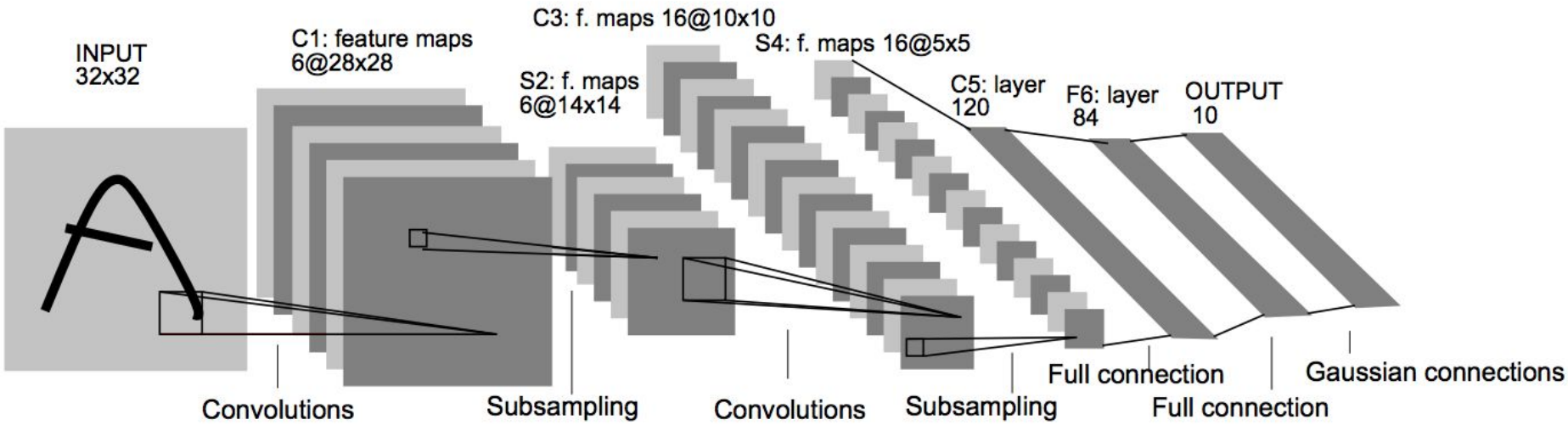
- Key Concept: GN is independent of batch size.
- Mechanism: It separates channels into groups (e.g., 32 channels -> 4 groups of 8). It calculates mean/variance for each group within a single sample.
- Why it works: Channels in CNNs often learn correlated features (e.g., different orientations of edges). Grouping them allows the model to normalize based on "feature types" rather than the whole layer.



History of CNNs

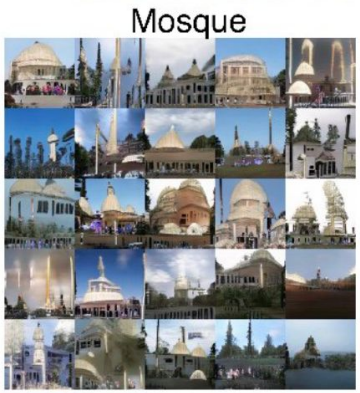
LeNet by Yann Lecun - 1998

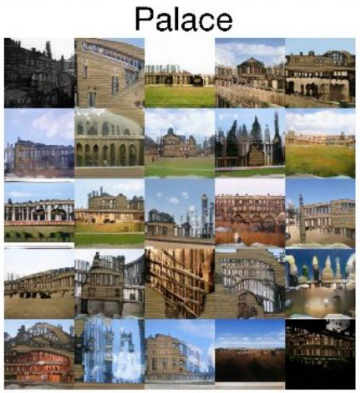
Parameters: 60,000

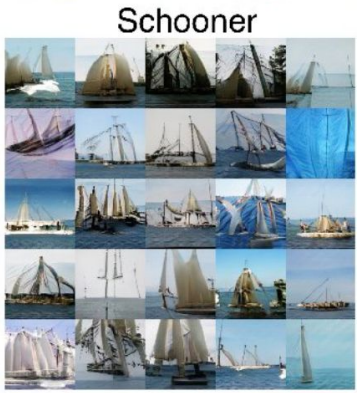


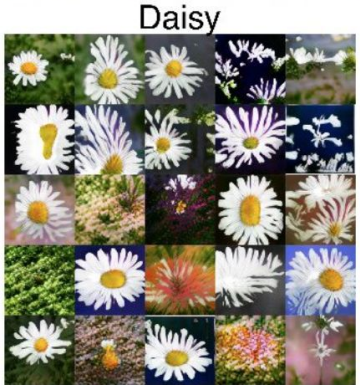
ImageNet dataset

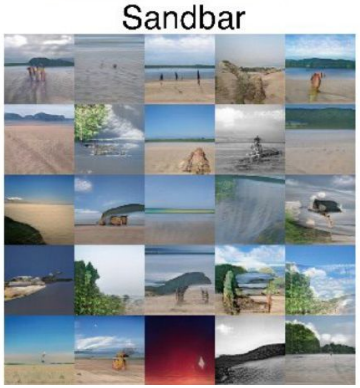
1000 classes and
1M+ images



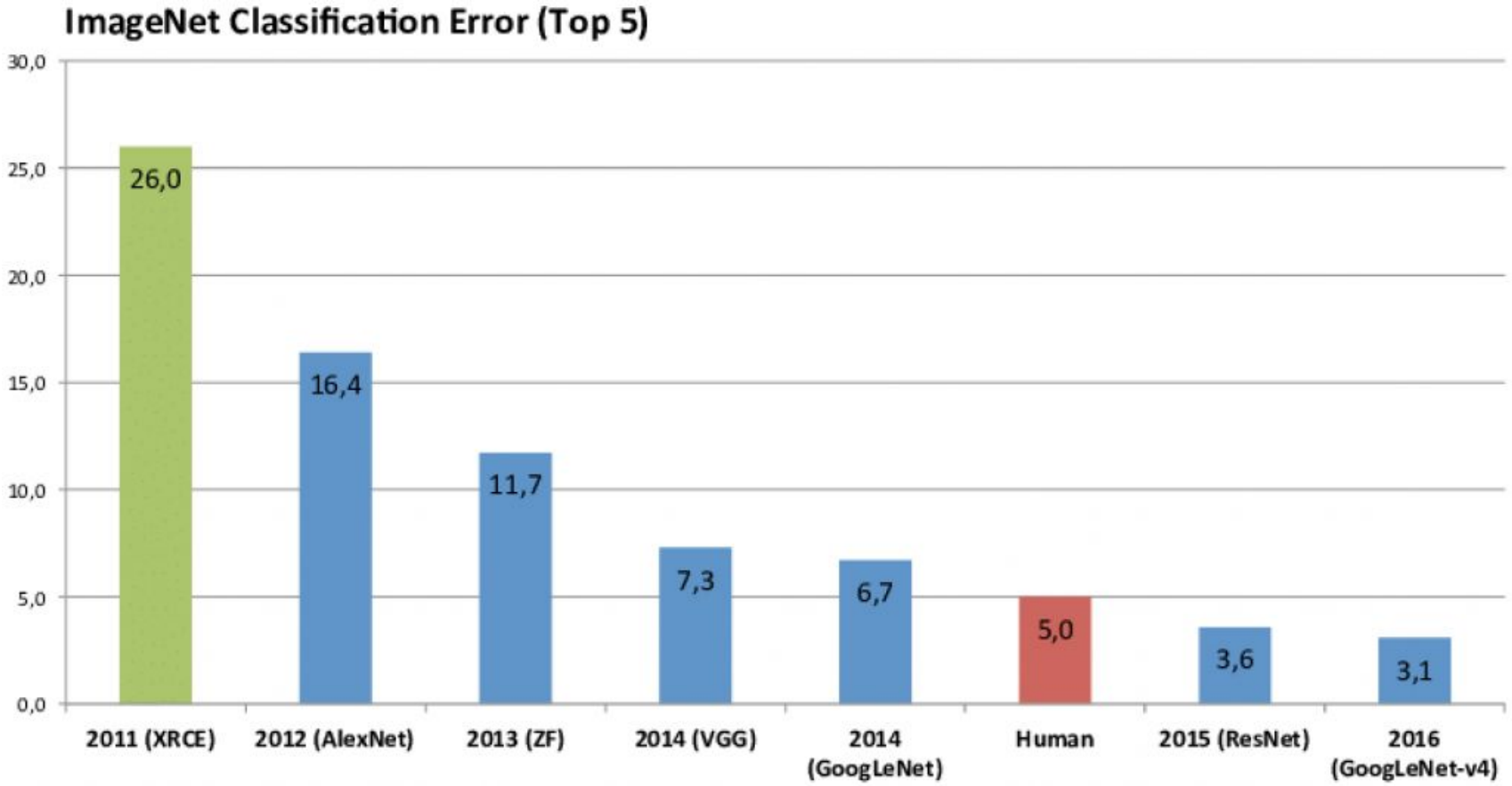






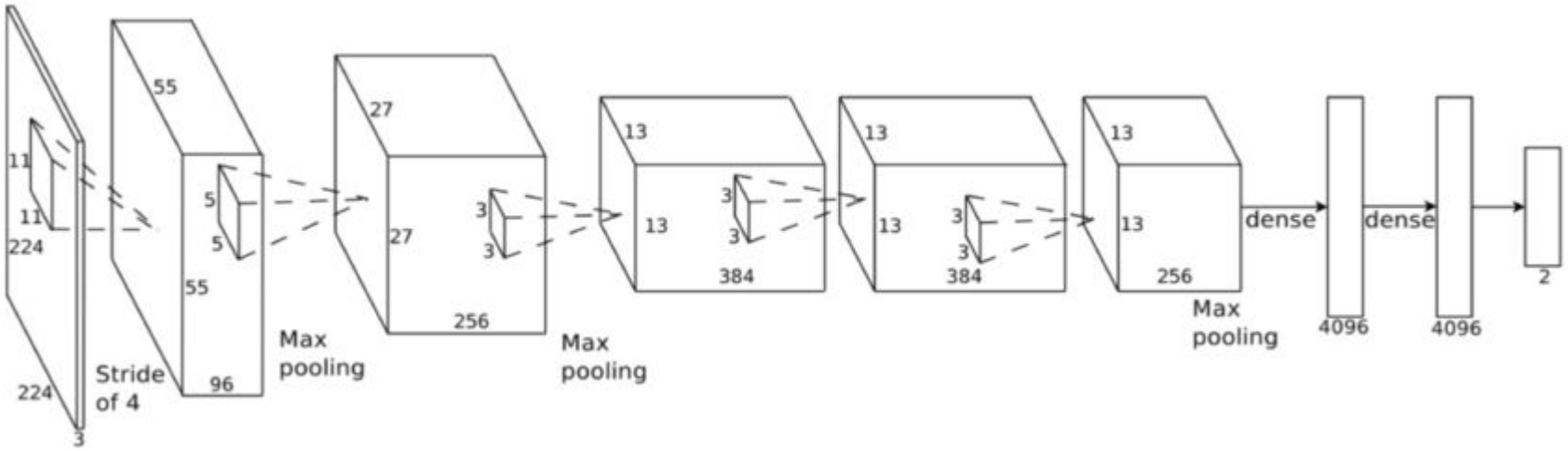


ImageNet Results

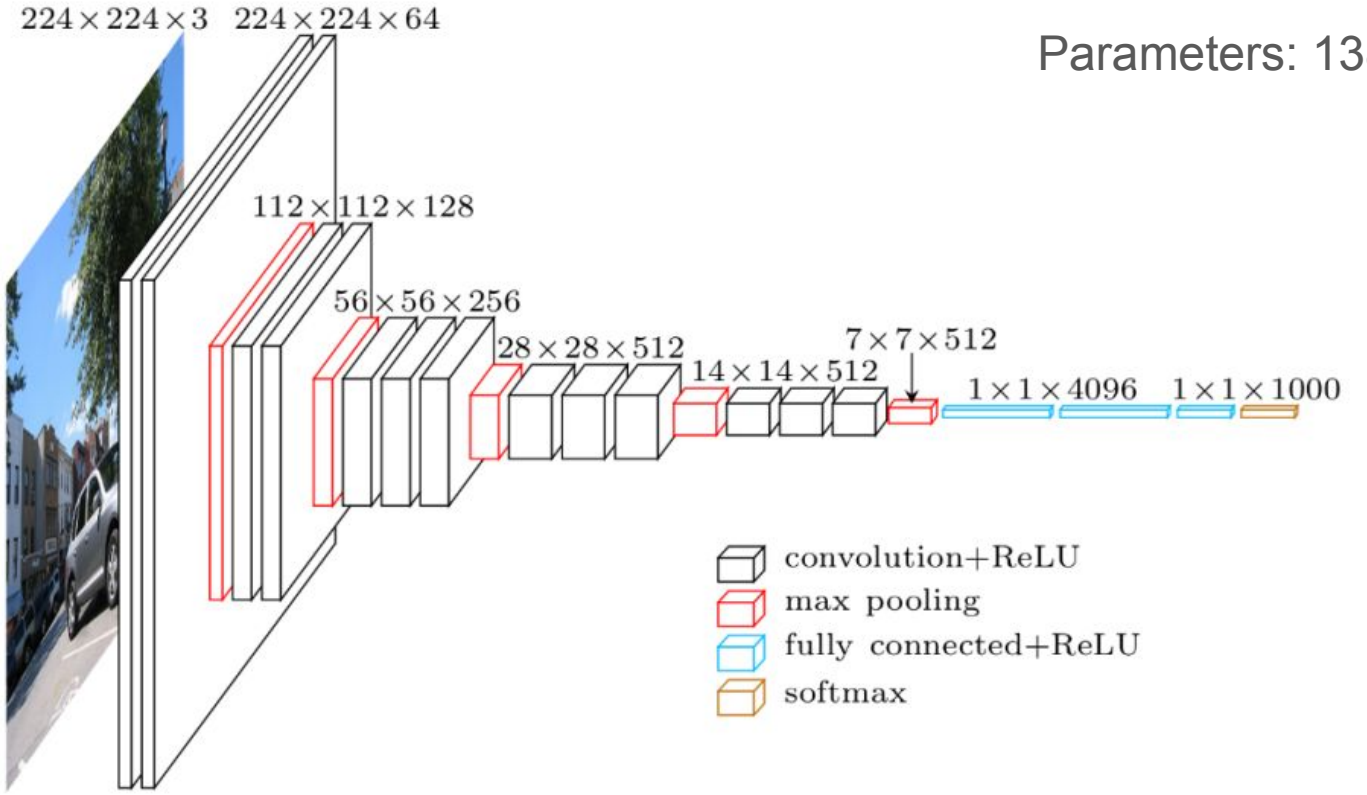


AlexNet by Alex Krizhevsky et al. in 2012

Parameters: 60 million



VGG by Karen Simonyan et al. in 2014



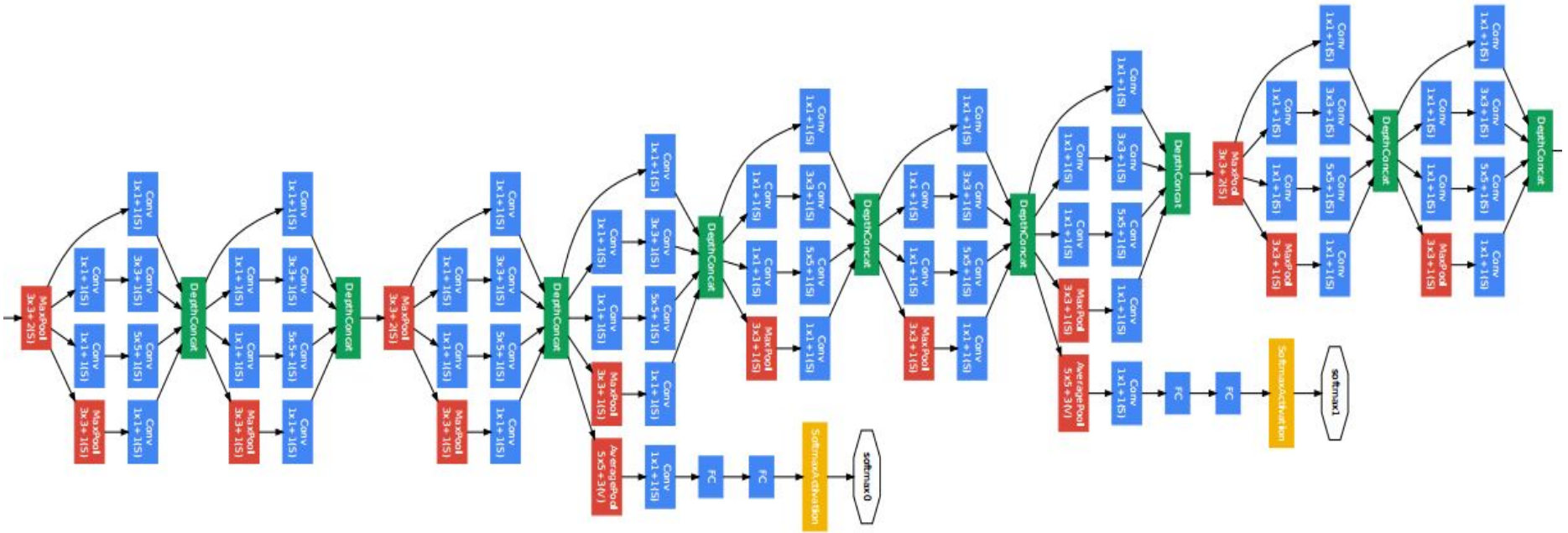
Parameters: 138 million

VGG-19

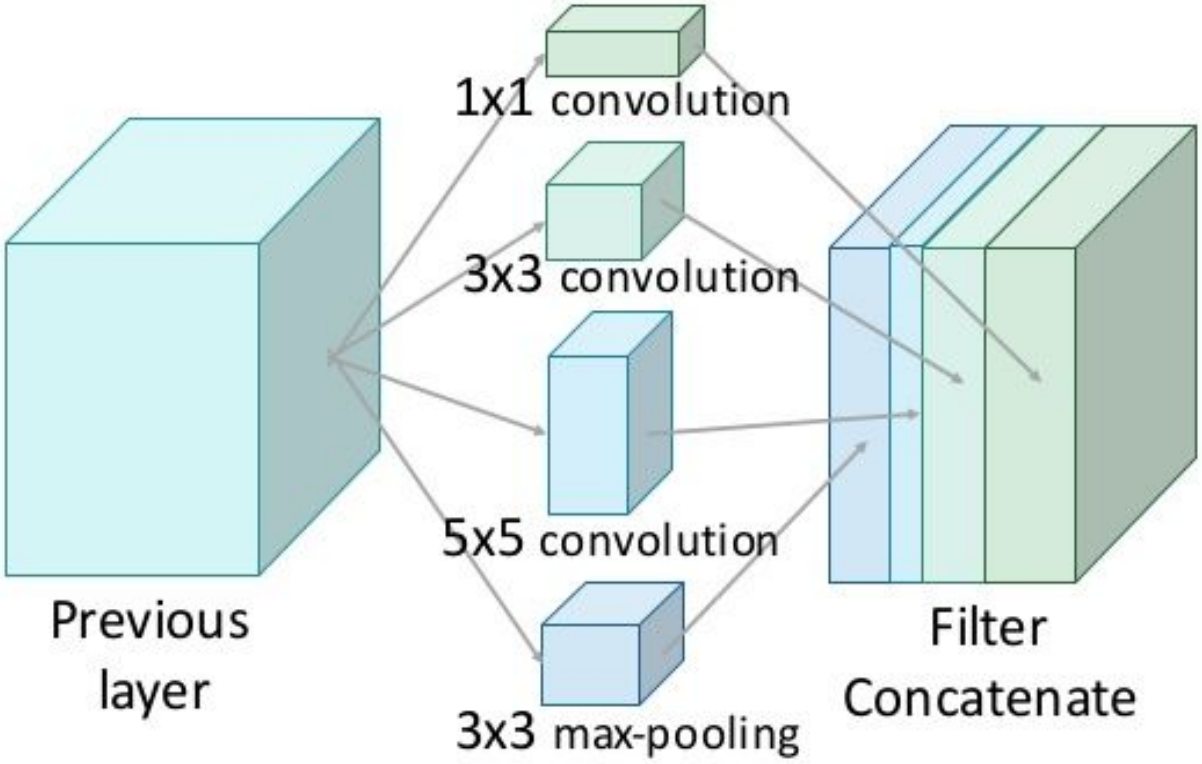
Top-1 Accuracy: 70.5%

Top-5 Accuracy: 90.0%

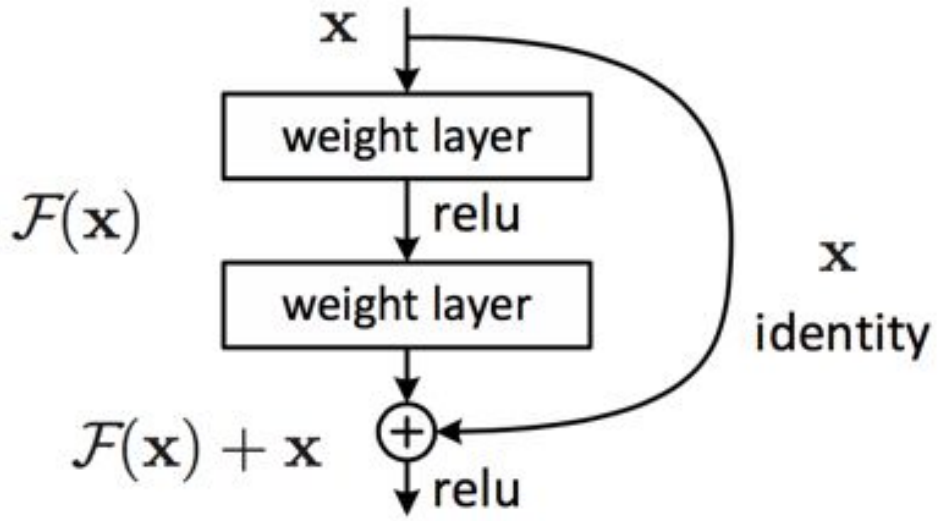
GoogleNet (InceptionNet)



InceptionNet Module

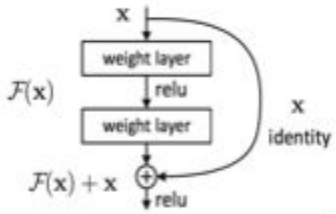


Residual Block (skip connection)

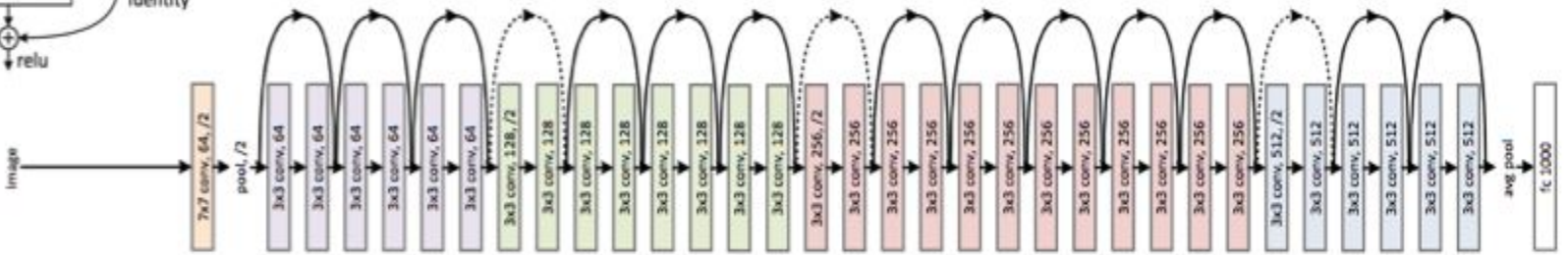


ResNet

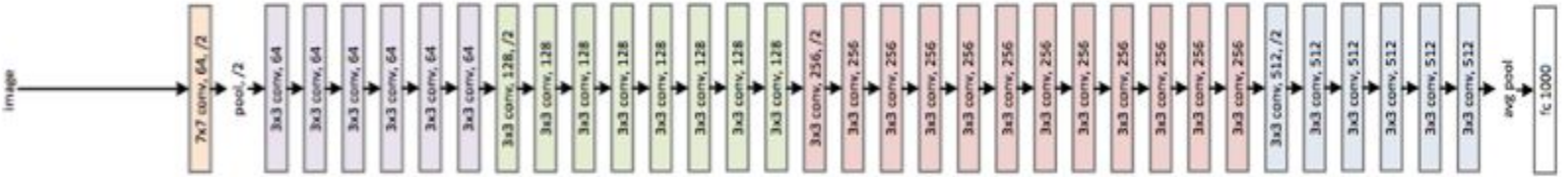
residual connection



Residual Network



Plain Network



L2 Regularization in CNNs

Key Concept: Penalizing large weights in Convolutional Kernels.

CNN Specifics:

- Without L2, kernels can become "noisy" or overly sharp to fit specific pixels.
- With L2: Kernels tend to be smoother and more distinct (Gabor-like filters).

Dropout in CNNs

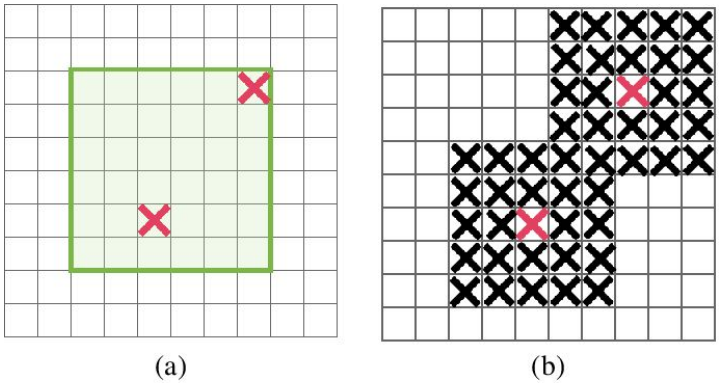
Key Concept: Standard Dropout turns off random neurons (pixels in feature maps).

The Problem with Standard Dropout in Conv Layers:

- Pixels in feature maps are spatially correlated. If you drop pixel (i, j), the network can still easily infer its value from neighbor (i, j+1).
- *Result:* Standard dropout is weak for Conv layers; it mostly just slows down training without much regularization benefit.

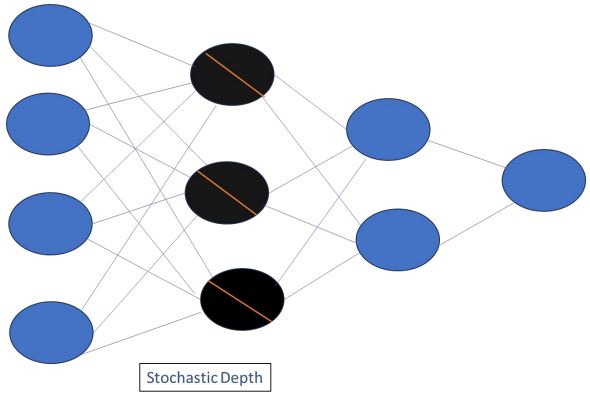
Dropout in CNNs

- **Key Concept:** Instead of dropping random pixels, drop entire *channels* or contiguous *blocks* of the feature map.
- **Spatial Dropout:** Drop the entire k-th feature map for a sample. Forces the network to not rely on that specific feature (e.g., "beak detector") at all.
- **DropBlock:** Drop a square region e.g., 5x5 block) of the feature map. Forces the network to look at other parts of the object.



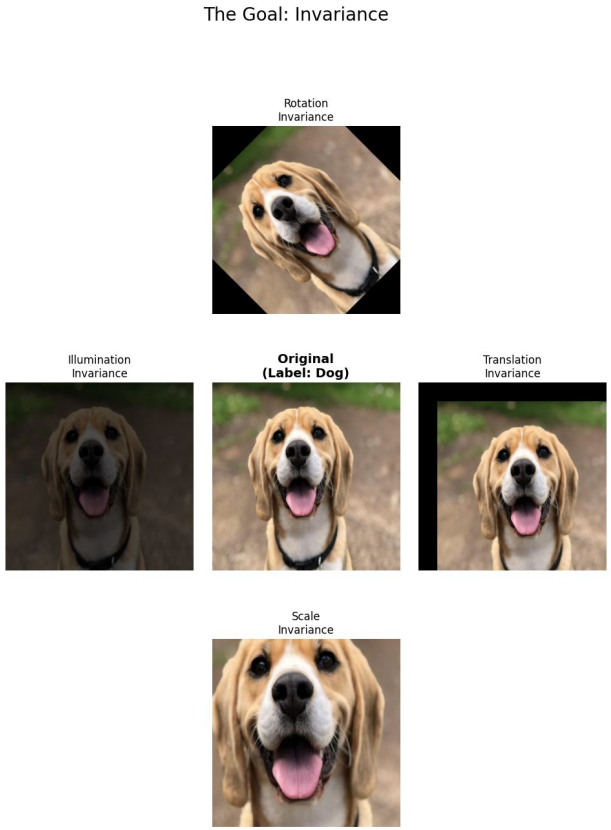
Stochastic Depth

- **Key Concept:** Used specifically for ResNets (in general applicable to only deep networks)
- **Mechanism:** Randomly "skip" entire Residual Blocks during training.
  a. Training: The network is effectively shallower (subset of layers).
  b. Testing: Use the full deep network.
- **Benefit:** Acts like an ensemble of many shallower networks.



Data Augmentation

- **Key Concept:** A "dog" is still a "dog" if it is on the left side, right side, or slightly darker.
- **Goal:** We want our CNN to be *invariant* to translation, viewpoint, size, and illumination.



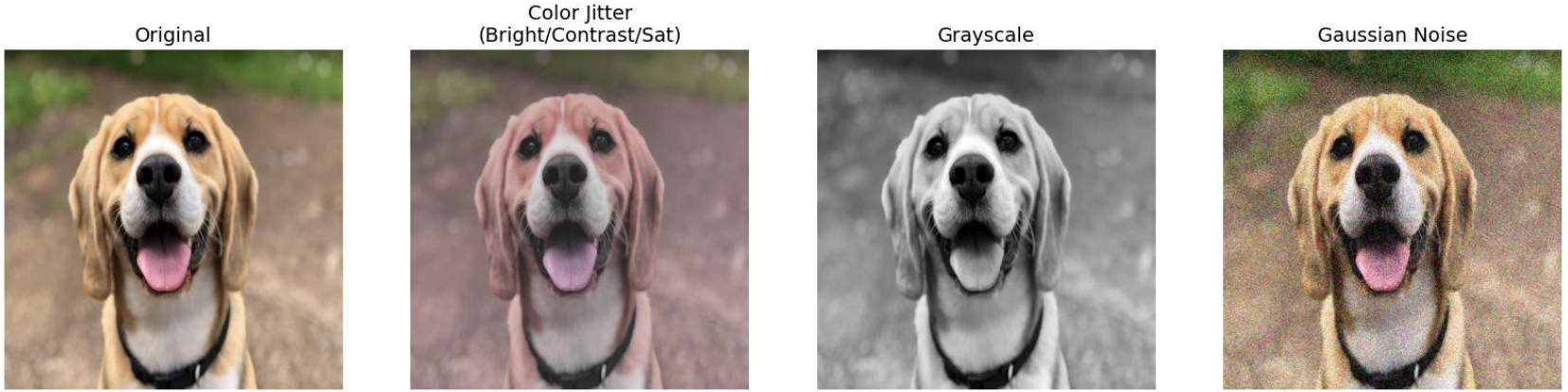
Geometric Augmentation

- **Key Concept:** Physically moving pixels.
- **Techniques:**
  a. **Random Resized Crop (The Gold Standard):** Pick a random patch of the image, resize it back to 224x224. This forces the network to learn "parts" of the object (e.g., just the ear) and the "whole".
  b. **Horizontal Flip:** Valid for natural scenes (not always valid for text/numbers!).

Random Resized Crop

Photometric Augmentations

- **Key Concept:** Changing pixel values, not positions.
- **Techniques:**
  - **Color Jitter:** Randomly change Brightness, Contrast, Saturation, Hue.
  - **Grayscale:** Randomly convert to B&W.
- **Why:** Prevents the network from relying on color alone (e.g., "strawberries are always red").



Advanced Augmentation

These are Modern "Strong" Augmentations (State-of-the-Art).

- **CutOut**: Randomly mask out a black square in the image. (Forces network to not look at just the most discriminative part).
- **MixUp**: Take two images (Cat, Dog). Linearly blend them: $0.7 \times \text{Cat} + 0.3 \times \text{Dog}$. Label becomes [0.7,0.3].
    - **Result**: The decision boundary becomes smoother/linear.
- **CutMix**: Cut a patch from a Cat image and paste it onto a Dog image. Label is proportional to area pixels.

Modern Augmentations

CutOut
(Region Dropout)

MixUp
(0.6*Dog + 0.4*Cat)

CutMix
(Patch Paste)

ImageNet Comparison

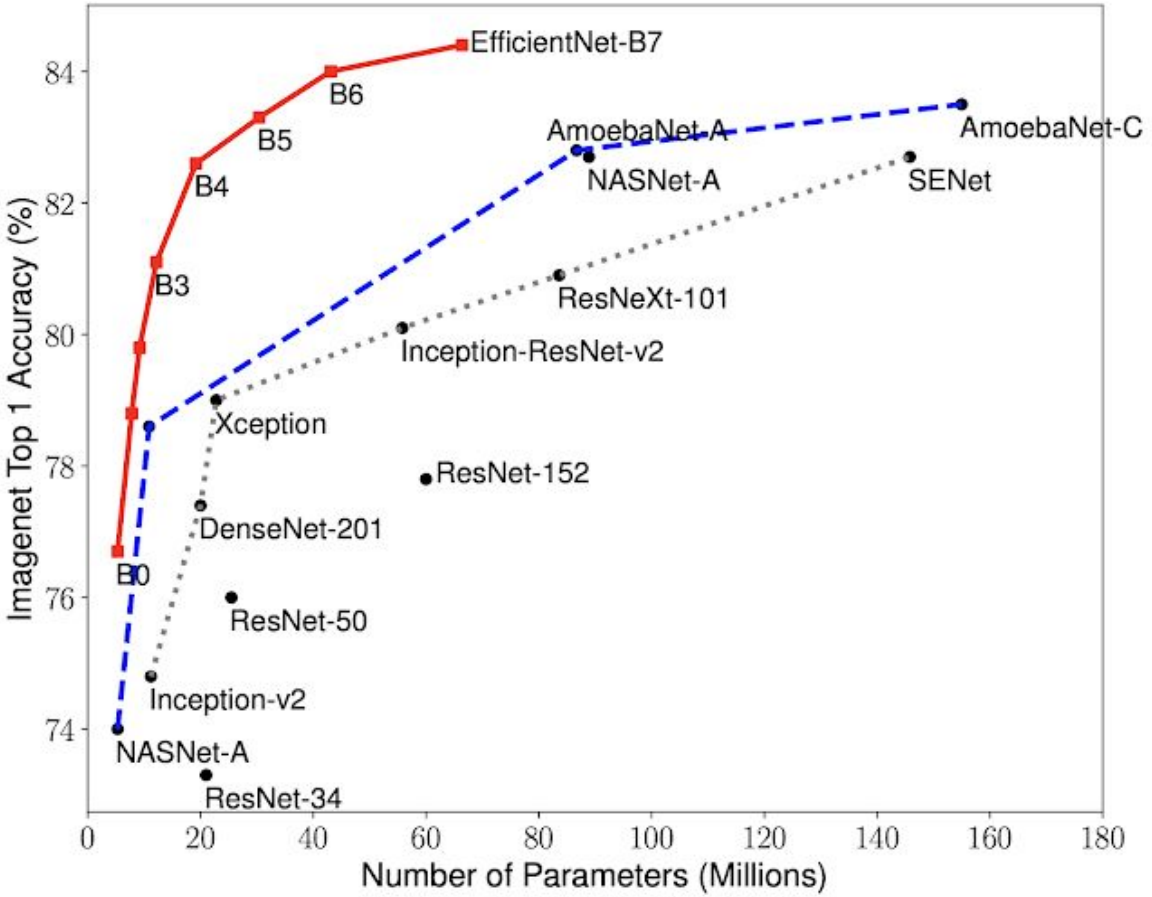
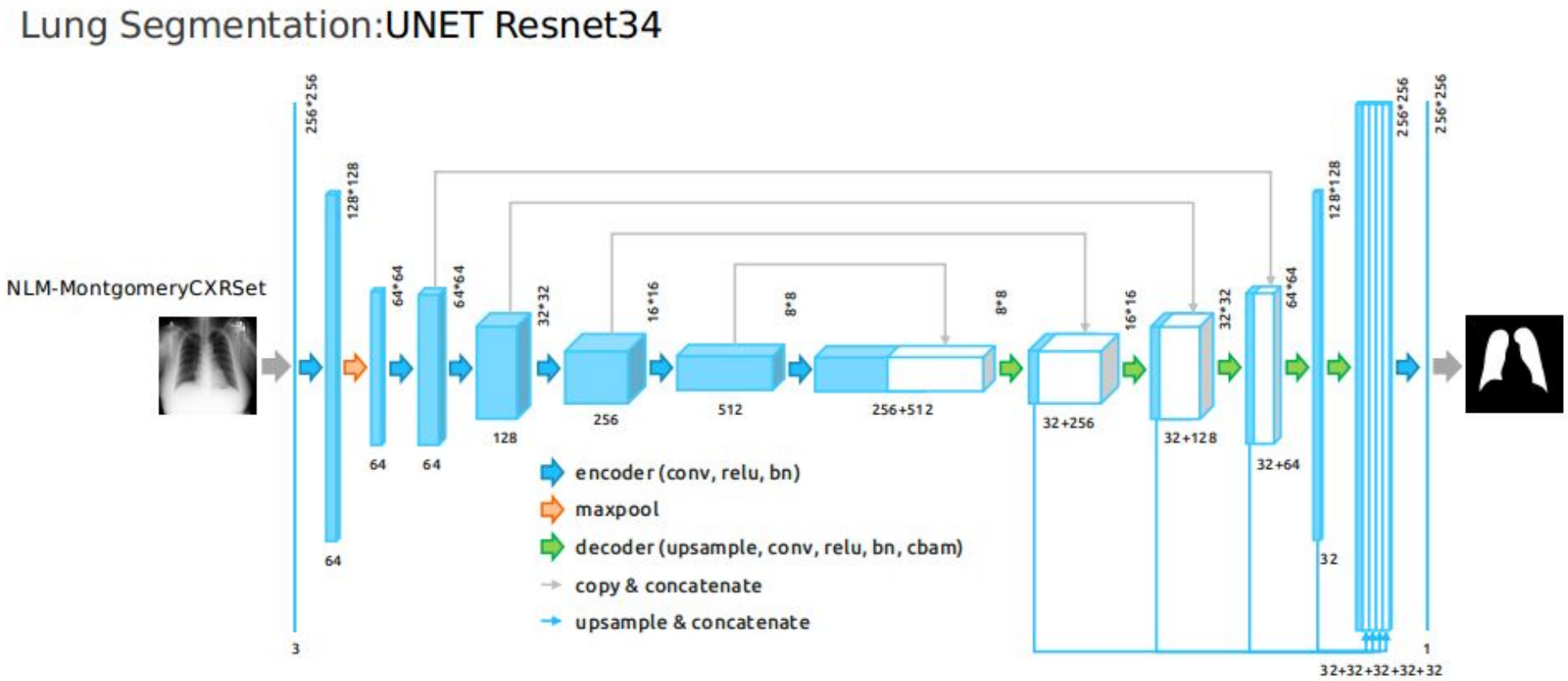
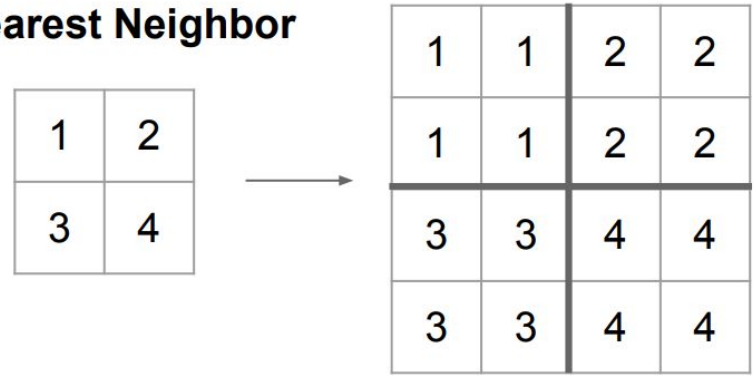


Image-to-image Network (U-Net)



In-Network Up-sampling: “Unpooling”

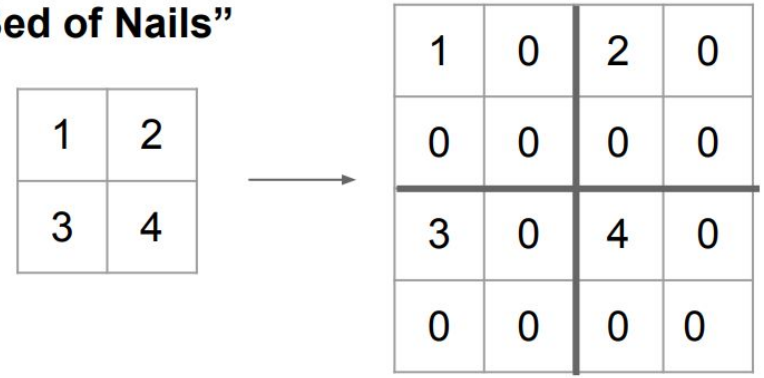
Nearest Neighbor



Input: 2 x 2

Output: 4 x 4

“Bed of Nails”



Input: 2 x 2

Output: 4 x 4

In-Network Up-sampling: “Max Unpooling”

Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

Rest of the network

Max Unpooling
Use positions from pooling layer

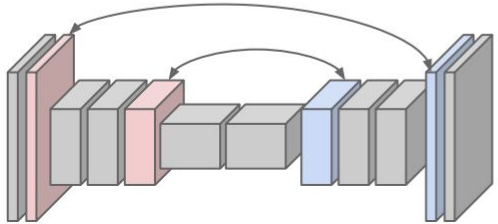
1	2
3	4

Input: 2 x 2

0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

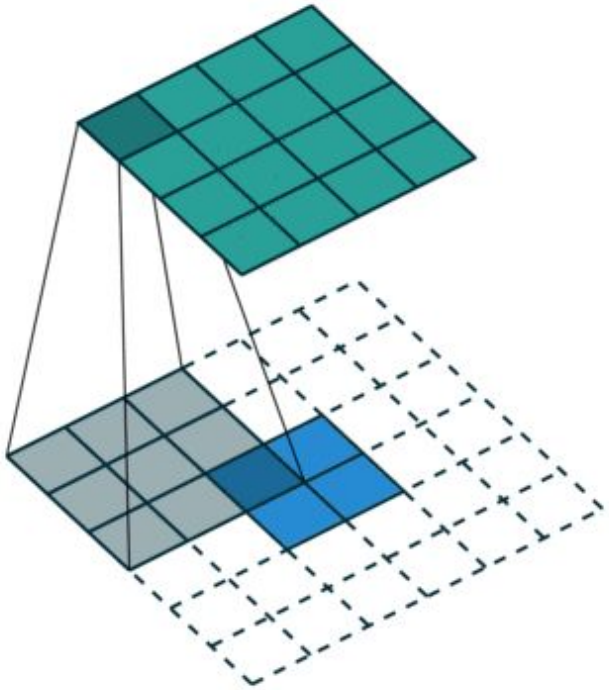
Output: 4 x 4

Corresponding pairs of downsampling and upsampling layers



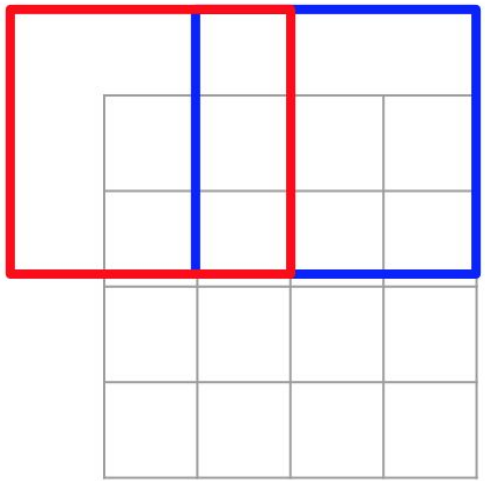
Learnable upsampling: transposed convolutions

Upconvolution is essentially the reverse operation of convolution. While convolution reduces the spatial dimensions of an input (e.g., image) to produce feature maps, upconvolution increases the spatial dimensions to produce a larger output.



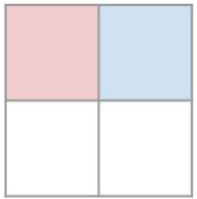
Convolution with Strides

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4

Dot product between filter and input

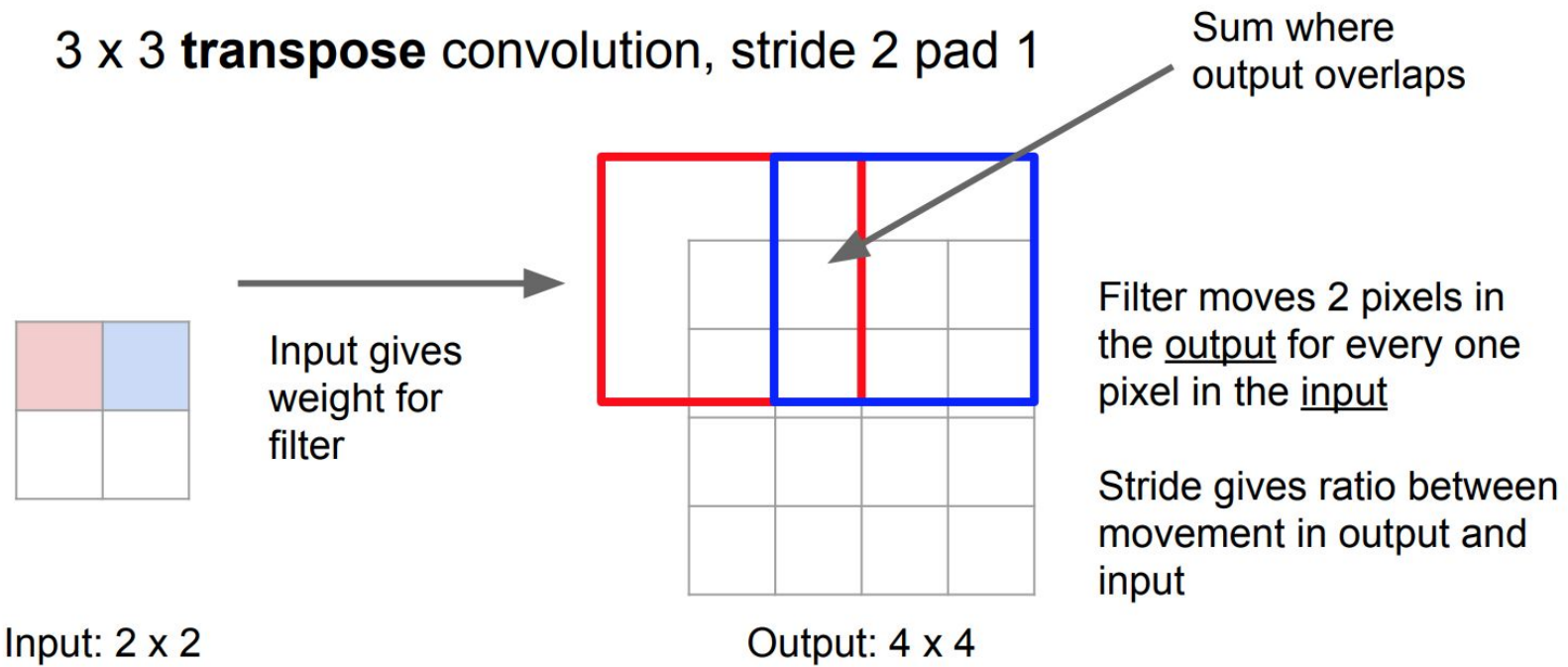


Output: 2 x 2

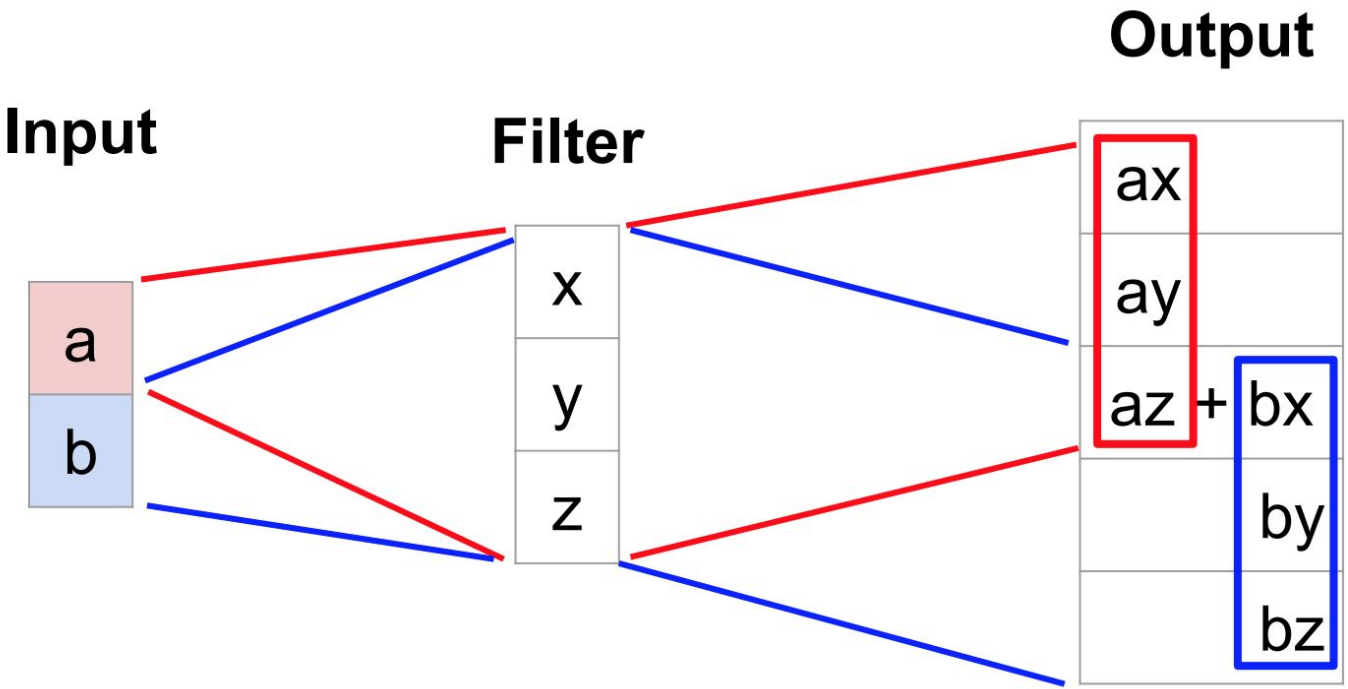
Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

Transposed convolutions



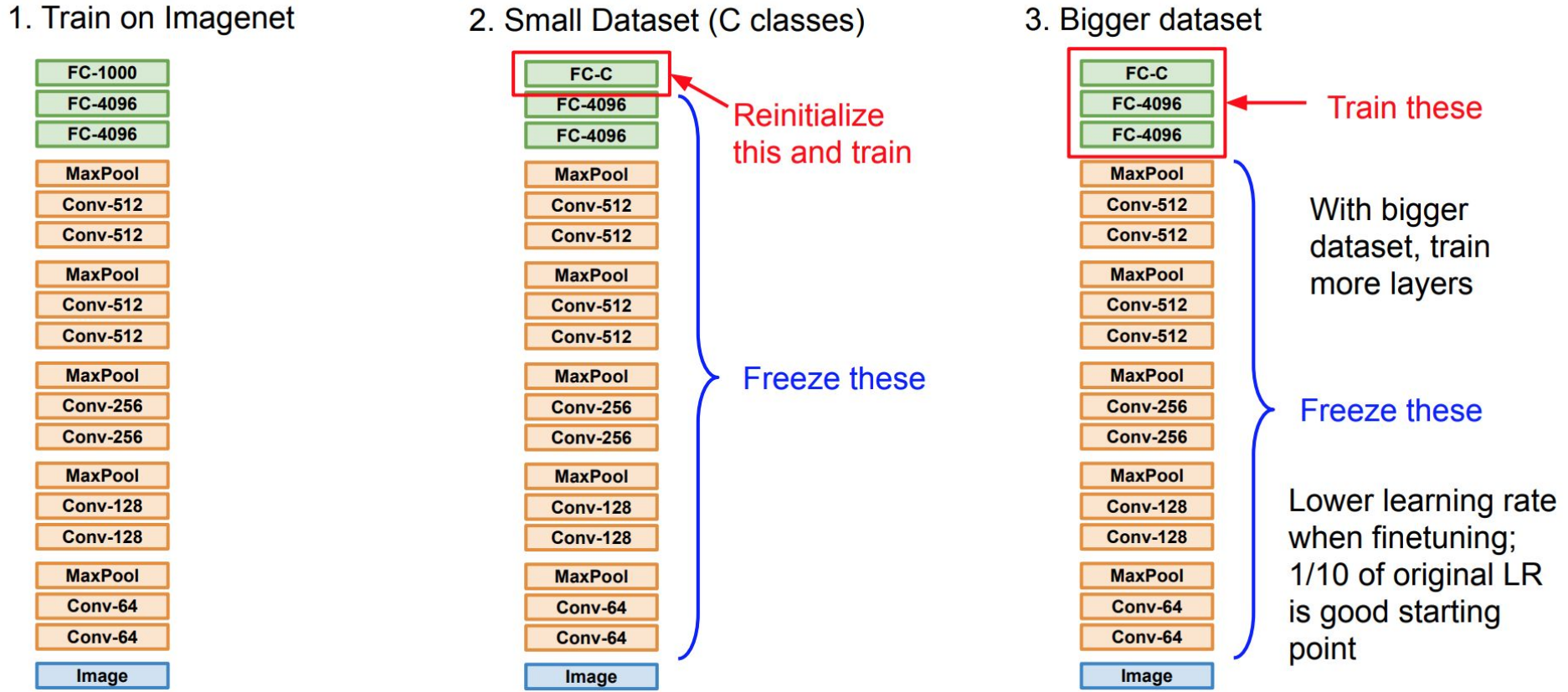
Transposed convolutions: 1D example



Output contains copies of the filter weighted by the input, summing at where at overlaps in the output

Need to crop one pixel from output to make output exactly 2x input

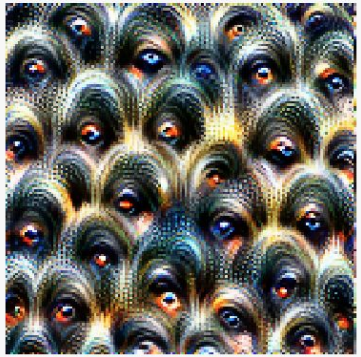
Transfer Learning



Feature Visualization

Demo

- Feature visualization
- Open AI Microscope



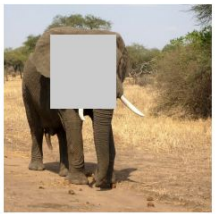
Positive optimized

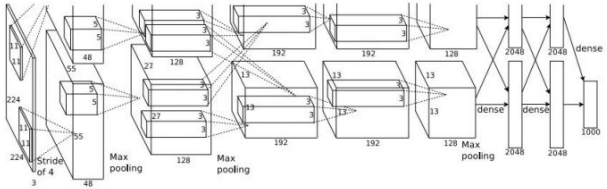
Maximum activation examples

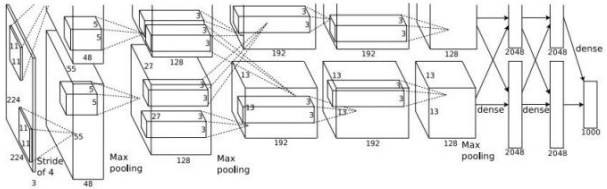
Slightly positive activation examples

Which Pixels Matter: Saliency vs Occlusion

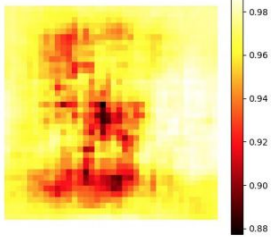
Mask part of the image before feeding to CNN, check how much predicted probabilities change



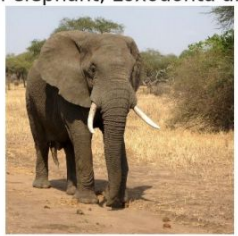


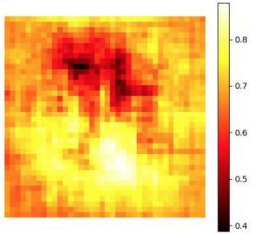


schooner

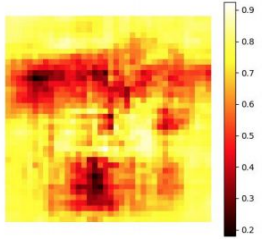


African elephant, Loxodonta africana





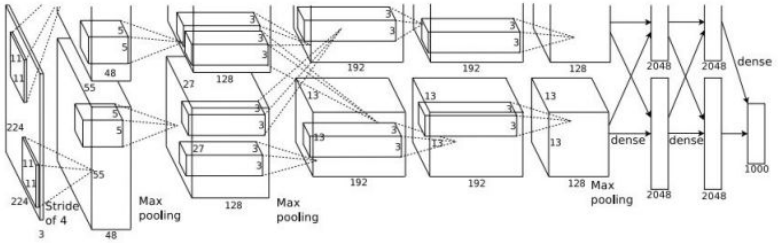
go-kart



Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

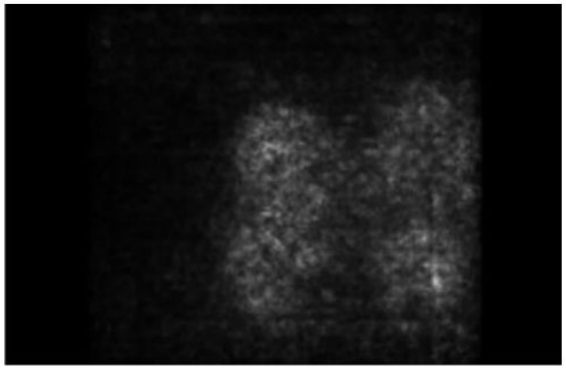
Which Pixels Matter: Saliency vs Backprop

Forward pass: Compute probabilities



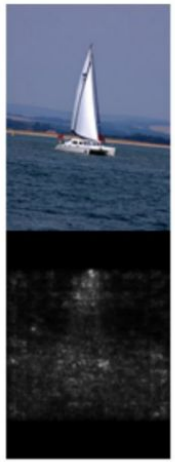
Dog

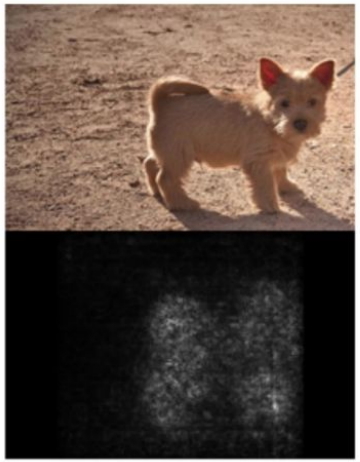
Compute gradient of (unnormalized) class score with respect to image pixels, take absolute value and max over RGB channels

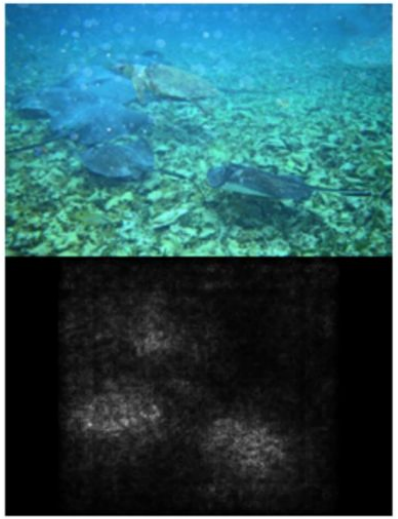


Simonyan, Vedaldi, and Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps", ICLR Workshop 2014.

Saliency Maps

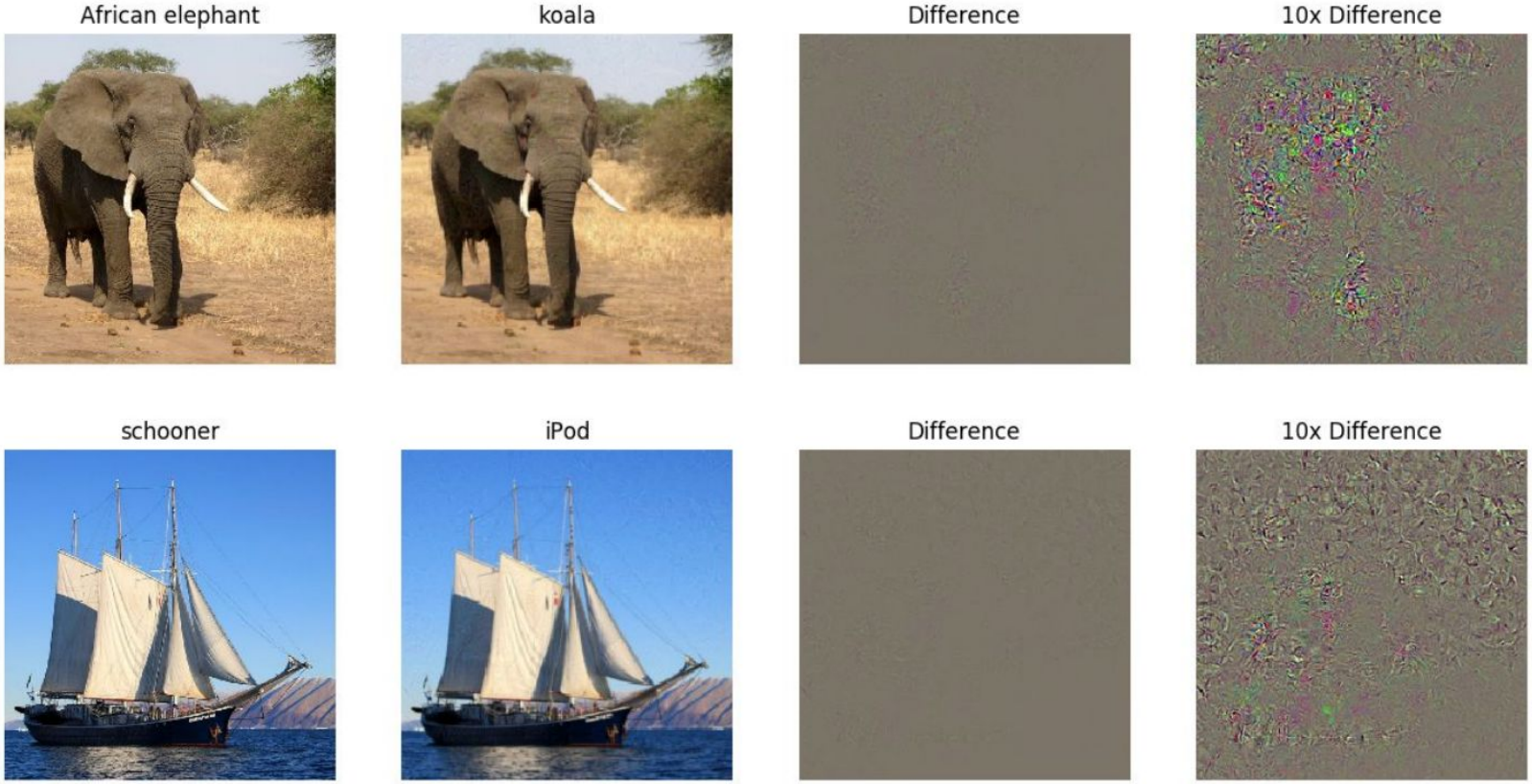






Simonyan, Vedaldi, and Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”, ICLR Workshop 2014.

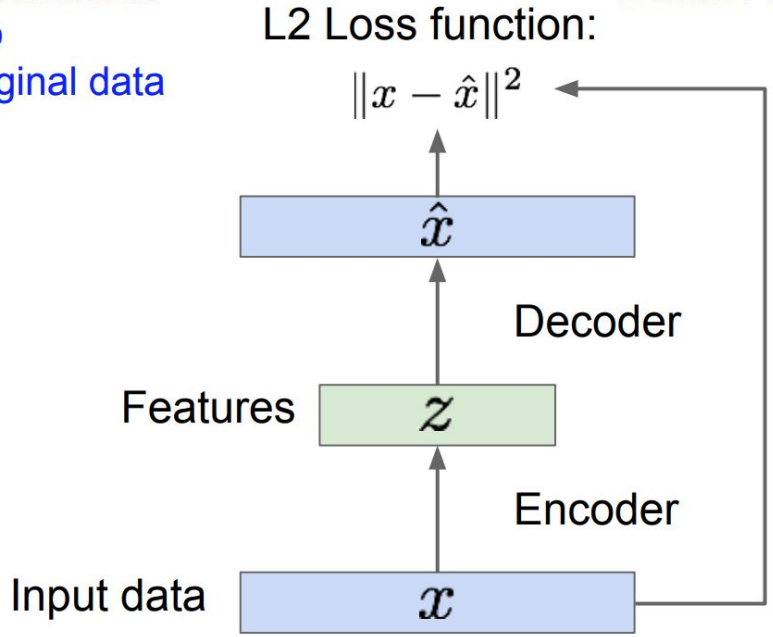
Adversarial examples

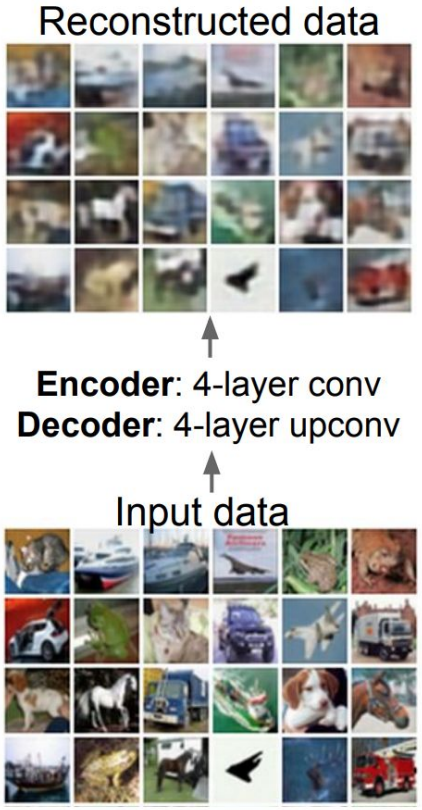


Autoencoders

Train such that features can be used to reconstruct original data

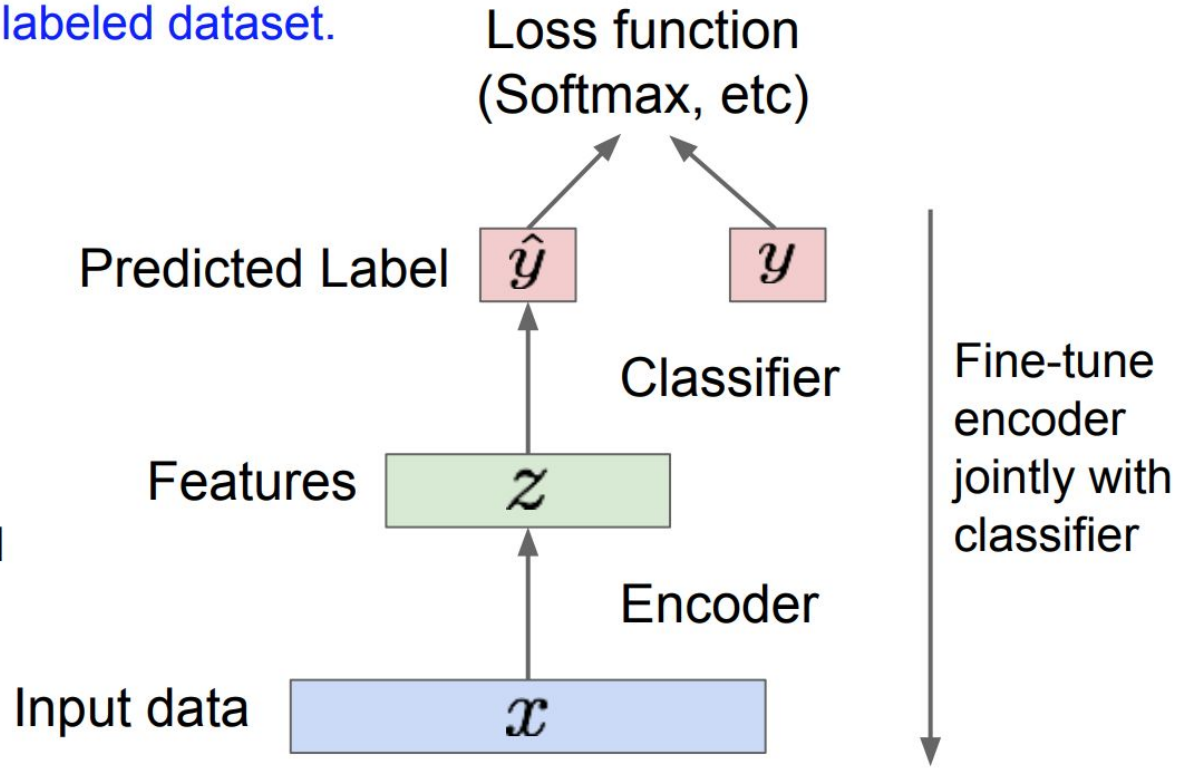
Doesn't use labels!





Autoencoders

Transfer from large, unlabeled dataset to small, labeled dataset.



Encoder can be used to initialize a **supervised** model